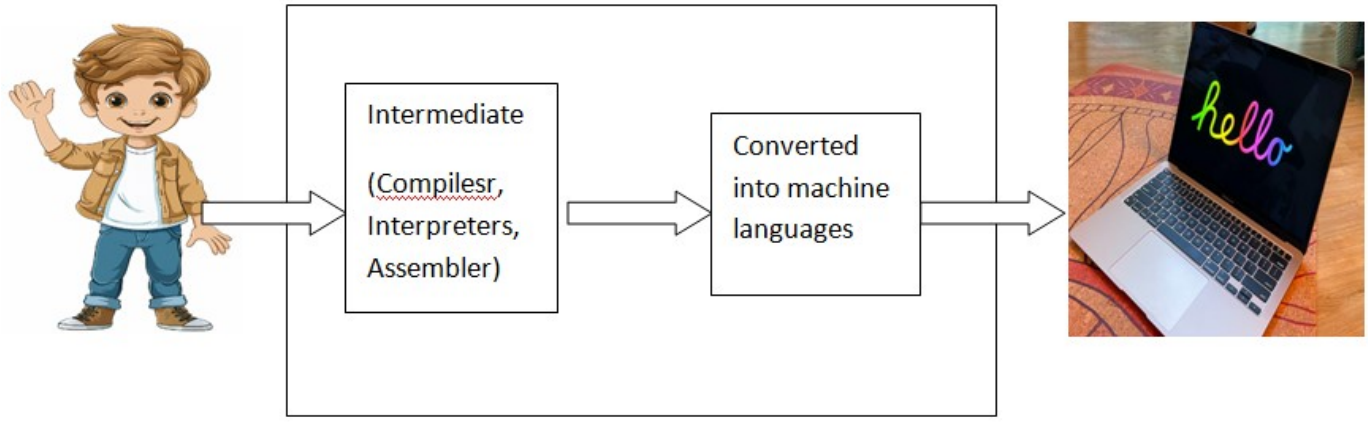
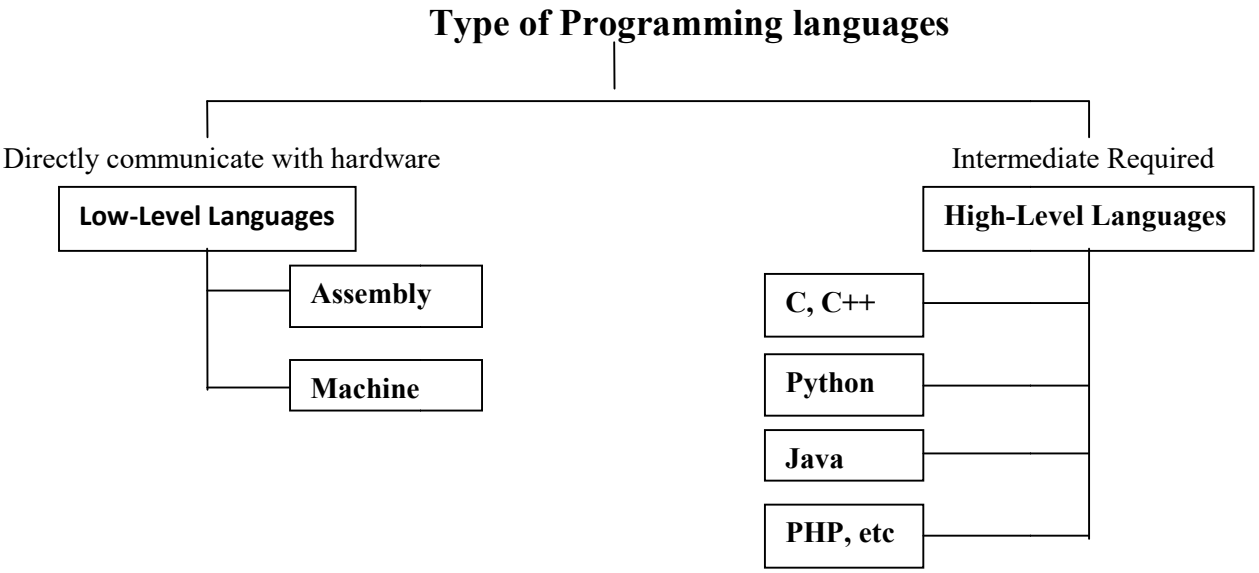
Language

Basics of languages

What is language ?
Why it is required ?
Type of languages ?
What is Low level languages (Machine & assembly language)?
What is high level languages ()?



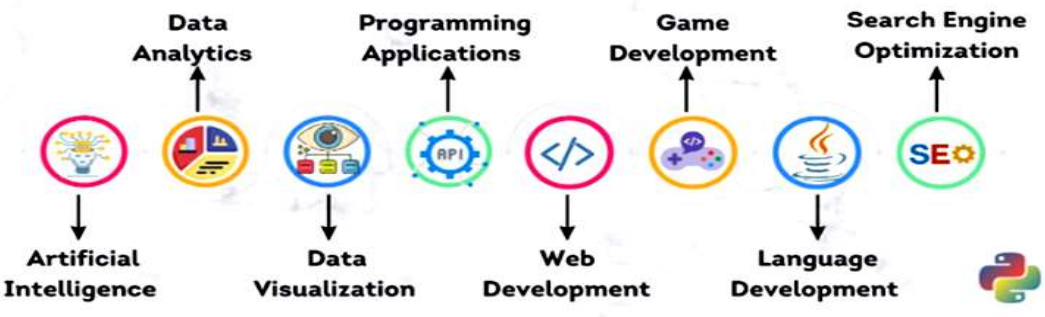


Python Introduction

Python is a :

1. Free and Open Source
2. General-purpose
3. High Level Programming language

That can be used for:



Features/Advantages of Python:

1. Simple and easy to learn
2. Procedure and object oriented
3. Platform Independent
4. Portable
5. Dynamically Typed

6. Both Procedure Oriented and Object Oriented
7. Interpreted
8. Vast Library Support

Syntex:----

Example 1:-

Java:

```
public class HelloWorld
{
        p s v main(String[] args)
{
        SOP("Hello world");
}
}
```

C:

```
#include<stdio.h>
void main()
{
        print("Hello world");
 }
```

Python:

```
print("Hello World")
```

Example 2:- To print the sum of 2 numbers

Java:

```
public class Add
{
public static void main(String[] args)
{
        int a,b;
        a =10;
        b=20;
        System.out.println("The Sum:"+(a+b));
}
}
```

C:

```
#include <stdio.h>
```

```
void main()
{
        int a,b;
         a =10;
         b=20;
        printf("The Sum:%d",(a+b));
}
```

Python:

```
A,b=10,20
print("The Sum:",(a+b))
```

Limitations of Python:

1. **Performance and Speed:** Python is an interpreted language, which means that it is slower than compiled languages like C or Java. This can be a problem for certain types of applications that require high performance, such as real-time systems or heavy computation.
2. **Support for Concurrency and Parallelism:** Python does not have built-in support for concurrency and parallelism. This can make it difficult to write programs that take advantage of multiple cores or processors.
3. **Static Typing:** Python is a dynamically typed language, which means that the type of a variable is not checked at compile time. This can lead to errors at runtime.
4. **Web Support:** Python does not have built-in support for web development. This means that programmers need to use third-party frameworks and libraries to develop web applications in Python
5. **Runtime Errors**

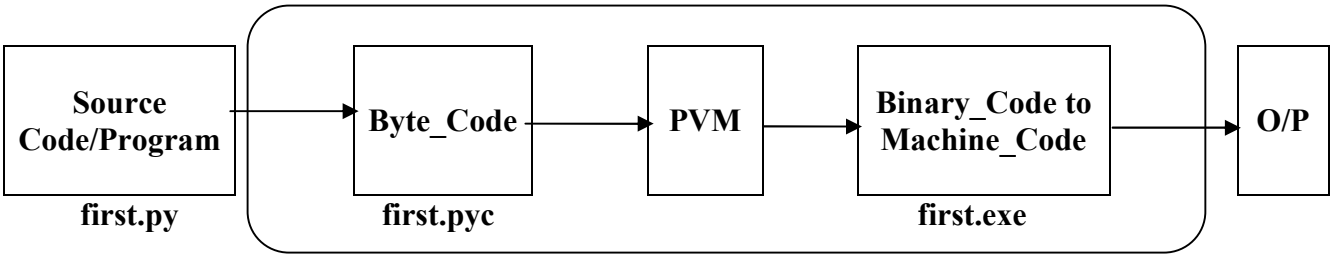
Python can take almost all programming features from different languages:--

1. Functional Programming Features from C
2. Object Oriented Programming Features from C++
3. Scripting Language Features from Perl and Shell Script
4. Modular Programming Features from Modula-3(Programming Language)

Flavors of Python or types of python compilers:

1. CPython:
It is the standard flavor of Python. It can be used to work with C lanugage Applications
2. Jython or JPython:
It is for Java Applications. It can run on JVM
3. IronPython:
It is for C#.Net platform
4. PyPy:
The main advantage of PyPy is performance will be improved because JIT (just in time)compiler is available inside PVM.
5. RubyPython
For Ruby Platforms
6. AnacondaPython
It is specially designed for handling large volume of data processing.

Internal working of python:-----



Examples:---

first.py:---

```
a = 10
b = 10
print("Sum ", (a+b))
```

The execution of the Python program involves 2 Steps:
- Compilation
- Interpreter

Compilation
The program is converted into **byte code.** Byte code is a fixed set of instructions that represent arithmetic, comparison, memory operations, etc. It can run on any operating

system and hardware. The byte code instructions are created in the **.pyc** file. The .pyc file is not explicitly created as Python handles it internally but it can be viewed with the following command:

```
PS E:\Python_data> python -m py_compile first.py
```

-m and py_compile represent module and module name respectively. This module is responsible to generate .pyc file. The compiler creates a directory named __pycache__ where it stores the first.cpython-310.pyc file.

(To convert a Python script to an executable file using PyInstaller, you can:

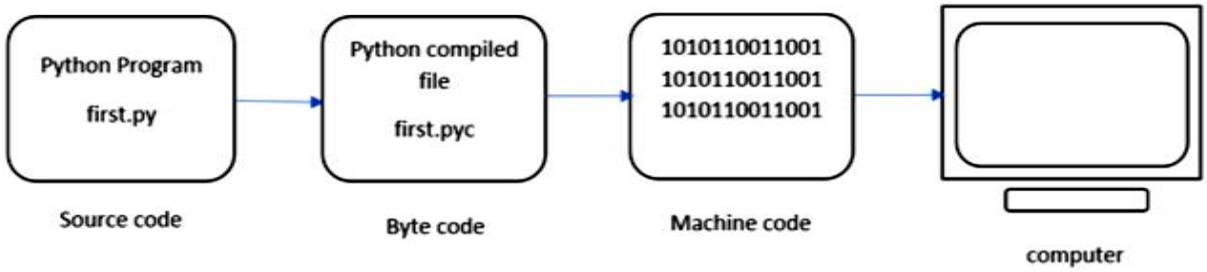
Install PyInstaller using pip: pip install pyinstaller

Navigate to the directory where your Python script is located

Run the following command: pyinstaller --onefile your_script.py

Interpreter

The next step involves converting the byte code (.pyc file) into machine code. This step is necessary as the computer can understand only machine code (binary code). Python Virtual Machine (PVM) first understands the operating system and processor in the computer and then converts it into machine code. Further, these machine code instructions are executed by processor and the results are displayed.



However, the interpreter inside the PVM translates the program line by line thereby consuming a lot of time. To overcome this, a compiler known as Just In Time (JIT) is added to PVM. JIT compiler improves the execution speed of the Python program. This compiler is not used in all Python environments like CPython which is standard Python software.

To execute the first.cpython-310.pyc we can use the following command:

```
PS E:\Python_data\__pycache__> python first1.cpython-310.pyc
```

view the byte code of the file – first.py we can type the following command as :

```
PS E:\DataSciencePythonBatch> python -m dis first.py
```

```
1           0 LOAD_CONST               0 (10)
            2 STORE_NAME               0 (a)

2           4 LOAD_CONST               0 (10)
            6 STORE_NAME               1 (b)

3           8 LOAD_NAME                2 (print)
           10 LOAD_CONST               1 ('Sum ')
           12 LOAD_NAME                0 (a)
           14 LOAD_NAME                1 (b)
           16 BINARY_ADD
           18 CALL_FUNCTION            2
           20 POP_TOP
           22 LOAD_CONST               2 (None)
           24 RETURN_VALUE
```

What is a variable in Python?

All the data which we create in the program will be saved in some memory location on the system. The data can be anything, an integer, a complex number, a set of mixed values, etc. A Python variable is a symbolic name that is a reference or pointer to an object. Once an object is assigned to a variable, you can refer to the object by that name.

To summarize, a variable is a

1. Name
2. Refers to a object
3. Hold some data

Rules for Creating Variables/Identifiers:---

1. A variable name must start with a letter or the underscore character.
   (myvar, my_var, _my_var, myVar, MYVAR, myvar2)

2. A variable name cannot start with a number.
   (2myvar,)

3. A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
   (myvar, my_var, _my_var, myVar, MYVAR, myvar2)

4. Variable names are case-sensitive.
   (age, Age and AGE are three different variables)

4. A variable name cannot be any of the python keywords.
   (if , else, for ,while, try etc)

   # how to fetch all python-keywords

   import keyword

   print("The list of keywords is : ")
   print(keyword.kwlist)

Assign Multiple Values in multiple variables in single line:-

1. Many Values to Multiple Variables
   Example:-
   x, y, z = "Neeraj", "Ravi", "Rahul"
   print(x)
   print(y)
   print(z)

2. **One Value to Multiple Variables in single line:**
   Example:-
   x = y = z = "Neeraj Kumar"
   print(x)
   print(y)
   print(z)

3. **Advance examples:-**
   Example:-
   city = ["Bhopal", "Indore", "Jabalpur"]
   x, y, z = city
   print(x)
   print(y)
   print(z)

Python Comments:-

1. single line comments:--- (# ----------) ctrl+/

2. Multi-line comments:---(‘‘‘ ----------
   ----------’’’)

Operator in Python

In programming languages, an operator is a symbol that is applied to some operands (usually variables), to perform certain actions or operations. For example,

```
x = 1
y = 2
z = x+y
print(z)
```

O/P:-

3

In the above example, first, we assigned values to two variables ‘a’ and ‘b’ and then we added both the variables and stored the result in variable ‘c’. The operations we performed are:

1. Assignment Operation using ‘=’ operator. We assigned the values to variables ‘a’, ‘b’, ‘c’ using the operator ‘=’ which is called the assignment operator.

2. Addition Operation using ‘+’ operator. The values in variables are added using the ‘+’ operator.

Note: The symbols + and = are called operators and the variables a,b,c on which operation is being performed are called operands.

BASIC CLASSIFICATION OF OPERATORS IN PYTHON

Unary Operator –A Unary Operator is a computational operator that takes any action on one operand and produces only one result. For example, the “-” binary

operator in Python turns the operand negative (if positive) and positive (if negative).
So, if the operator acts on a single operand then it's called the unary operator. For example, in '-5' the operator '-' is used to make the (single) operand '5' a negative value hence called the unary operator. **Ex:-1**

```
# Unary Operator.
x=10
y=-(x)
print("For - unary operator:",y)
x=10
y=+(x)
print("For + unary operator:",y)
```

O/P:

```
-10
10
```

Binary Operator – If the operator acts on two operands then it's called a binary operator. For example, + operators need two variables (operands) to perform some operation, hence called binary operator.

```
# Binary Operator
# for + operator
x = 5
y = 4
z = x+y
print("For + binary operator:",z)
# for - operator
x = 5
y = 4
z = x-y
print("For - binary operator:",z)
# for * operator
x = 5
y = 4
z = x*y
print("For * binary operator:",z)
# for / operator
x = 5
y = 4
z = x/y
print("For / binary operator:",z)
```

```
# for %(Modulus) operator
x = 5
y = 4
z = x%y
print("For % binary operator:",z)
# for //(floor division) operator
x = 5
y = 4
z = x//y
print("For - binary operator:",z)
```

O/P:-

```
For + binary operator: 9
For - binary operator: 1
For * binary operator: 20
For / binary operator: 1.25
For % binary operator: 1
For - binary operator: 1
```

Ternary Operator – If the operator acts on three operands then it's called a ternary operator. In general, the ternary operator is a precise way of writing conditional statements.

Syntax:

x = true_value if **condition** else **false_value**

The three operands here are:

1. condition
2. true_value
3. False_value

```
# Ternary Operator
a, b = 10, 20
min = a if a < b else b
print("Ternary Operator(min): ",min)
```

O/P:- Ternary Operator(min): 10

Types Of Operators in Python:

1. **Arithmetic Operators:** The operators which are used to perform arithmetic operations like addition, subtraction, division etc.
(+, -, *, /, %, **, //)

2. Relational Operators/Comparison Operators: The operators which are used to check for some relation like greater than or less than etc.. between operands are called relational operators.

(<, >, <=, >=, ==, !=)

3. **Logical Operators:** The operators which do some logical operation on the operands and return True or False are called logical operators. The logical operations can be **'and', 'or', 'not'** etc.

4. **Assignment Operators:** The operators which are used for assigning values to variables. **'='** is the assignment operator.

5. **Unary Operator:** The operator '-' is called the Unary minus operator. It is used to negate the number.

6. Membership Operators: The operators that are used to check whether a particular variable is part of another variable or not.
('in' and 'not in')

7. **Identity Operators:** The operators which are used to check for identity.
('is' and 'is not')

8. Python Bitwise Operators: Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

ARITHMETIC OPERATORS IN PYTHON:

As stated above, these are used to perform that basic mathematical stuff as done in every programming language. Let's understand them with some examples. Let's assume, a = 20 and b = 12

Operator	Meaning	Example	Result
+	Addition	a+b	32
-	Subtraction	a-b	8
*	Multiplication	a*b	240
/	Division (Quotient of the division)	a/b	1.6666666666666667
%	Modulus (Remainder of division)	a%b	8
**	Exponent operator	a**b	4096000000000000
//	Integer division(gives only integer quotient)	a//b	1

Note: Division operator / always performs floating-point arithmetic, so it returns a float value. Floor division (//) can perform both floating-point and integral as well,

1. If values are int type, the result is int type.
2. If at least one value is float type, then the result is of float type.

Example: Arithmetic Operators in Python:

```
a = 20
b = 12
print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a%b)
print(ab)
print(a//b)
```

```
O/P:-
32
8
240
1.6666666666666667
8
4096000000000000
1
```

Example: Floor division

```
print(12//5)
print(12.0//5)

O/P:-
2
2.0
```

Relational Operators in Python:-

These are used to compare two values for some relation and return True or False depending on the relation. Let's assume, a = 13 and b = 5.

Operator	Example	Result
>	a>b	True
>=	a>=b	True
<	a<b	False
<=	a<=b	False
==	a==b	False
!=	a!=b	True

Example: Relational Operators in Python

```
a = 13
b = 5
print(a>b)
```

```
print(a>=b)
print(a<b)
print(a<=b)
print(a==b)
print(a!=b)

O/P:-
True
True
False
False
False
True
```

LOGICAL OPERATORS IN PYTHON:-

In python, there are three types of logical operators. They are and, or, not. These operators are used to construct compound conditions, combinations of more than one simple condition. Each simple condition gives a boolean value which is evaluated, to return the final boolean value.

Note: In logical operators, False indicates 0(zero) and True indicates non-zero value. Logical operators on boolean types

1. and: If both the arguments are True then only the result is True
2. or: If at least one argument is True then the result is True
3. not: the complement of the boolean value

Example: Logical operators on boolean types in Python

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
print(a and a)

O/P:-
False
True
False
True
```

and operator:

‘A and B’ returns A if A is False

‘A and B’ returns B if A is not False

Or Operator in Python:

‘A or B’ returns A if A is True

‘A or B’ returns B if A is not True

Not Operator in Python:

not A returns False if A is True

not B returns True if A is False

ASSIGNMENT OPERATORS IN PYTHON

By using these operators, we can assign values to variables. ‘=’ is the assignment operator used in python. There are some compound operators which are the combination of some arithmetic and assignment operators (+=, -=, *=, /=, %=, **=, //=). Assume that, a = 13 and b = 5

Operator	Example	Equal to	Result
=	x = a + b	x = a + b	18
+=	a += 5	a = a + 5	18
-=	a -= 5	a = a - 5	8

Example: Assignment Operators in Python

```
a=13
print(a)
a+=5
print(a)

O/P:-
13
18
```

UNARY MINUS OPERATOR(-) IN PYTHON:

This operator operates on a single operand, hence unary operator. This is used to change a positive number to a negative number and vice-versa.

Example: Unary Minus Operators in Python

```
a=10
print(a)
print(-a)

O/P:-
10
-10
```

MEMBERSHIP OPERATORS IN PYTHON

Membership operators are used to checking whether an element is present in a sequence of elements are not. Here, the sequence means strings, list, tuple, dictionaries, etc which will be discussed in later chapters. There are two membership operators available in python i.e. in and not in.

1. **in operator:** The in operators returns True if element is found in the collection of sequences. returns False if not found
2. **not in operator:** The not-in operator returns True if the element is not found in the collection of sequence. returns False in found

Example: Membership Operators in Python

```
text = "Welcome to python programming"
print("Welcome" in text)
print("welcome" in text)
print("nireekshan" in text)
print("Hari" not in text)

O/P:-
True
False
False
True
```

Example: Membership Operators in Python

```
names = ["Ramesh", "Nireekshan", "Arjun", "Prasad"]
print("Nireekshan" in names)
print("Hari" in names)
print("Hema" not in names)

O/P:-
True
False
True
```

IDENTITY OPERATOR IN PYTHON

This operator compares the memory location(address) to two elements or variables or objects. With these operators, we will be able to know whether the two objects are pointing to the same location or not. The memory location of the object can be seen using the id() function.

Example: Identity Operators in Python

```
a = 25
b = 25
print(id(a))
print(id(b))

O/P:-
1487788114928
1487788114928
```

Types of Identity Operators in Python:

There are two identity operators in python, is and is not.

is:

1. A is B returns True, if both A and B are pointing to the same address.
2. A is B returns False, if both A and B are not pointing to the same address.

is not:

1. A is not B returns True, if both A and B are not pointing to the same object.
2. A is not B returns False, if both A and B are pointing to the same object.

Example: Identity Operators in Python

```
a = 25
b = 25
print(a is b)
print(id(a))
print(id(b))

O/P:-
True
2873693373424
2873693373424
```

Example: Identity Operators

```
a = 25
b = 30
print(a is b)
print(id(a))
print(id(b))

O/P:-
False
1997786711024
1997786711184
```

Note: The 'is' and 'is not' operators are not comparing the values of the objects. They compare the memory locations (address) of the objects. If we want to compare the value of the objects. we should use the relational operator '=='.

Example: Identity Operators

```
a = 25
b = 25
print(a == b)

O/P:-
True
```

Input and Output in Python

Why it is required?

Example: Hardcoded values to a variable:

```
age = 18
if age>=18:
    print("eligible for vote")
else:
    print("Not eligible for vote")


age = 17
if age>=18:
    print("eligible for vote")
else:
    print("Not eligible for vote")

O/P:-
eligible for vote
Not eligible for vote
```

A predefined function input() is available in python to take input from the keyboard during runtime. This function takes a value from the keyboard and returns it as a string type. Based on the requirement we can convert from string to other types.

Now, if we want to take value of age at runtime, then we use python-inbuilt function **input().**

```
age=input("Enter Your age: ")
if age>=18:
    print("eligible for vote")
else:
    print("Not eligible for vote")

O/P:-
Enter Your age: 15
```

```
Traceback (most recent call last):
  File "E:\DataSciencePythonBatch\input_output.py", line 16, in <module>
    if age>=18:
TypeError: '>=' not supported between instances of 'str' and 'int'
```

We can convert the string to other data types using some inbuilt functions. We shall discuss all the type conversion types in the later chapters. As far as this chapter is concerned, it's good to know the below conversion functions.

1. **string to int – int() function**
2. **string to float – float() function**

```
age=input("Enter Your age: ")
print(type(age))
age=int(age)
print(type(age))
if age>=18:
    print("eligible for vote")
else:
    print("Not eligible for vote")

O/P:-
Enter Your age: 15
<class 'str'>
<class 'int'>
Not eligible for vote
```

```
# perform operations through input function.
x=int(input("Enter first No: "))
y=int(input("Enter second No: "))

z= x+y
print("Addition of x and y :",z)

O/P:-
Enter first No: 5
Enter second No: 4
Addition of x and y : 9
```

Eval() function in python:---

This is an in-built function available in python, which takes the strings as an input. The strings which we pass to it should, generally, be expressions. The eval() function takes the expression in the form of a string and evaluates it and returns the result.

Examples,

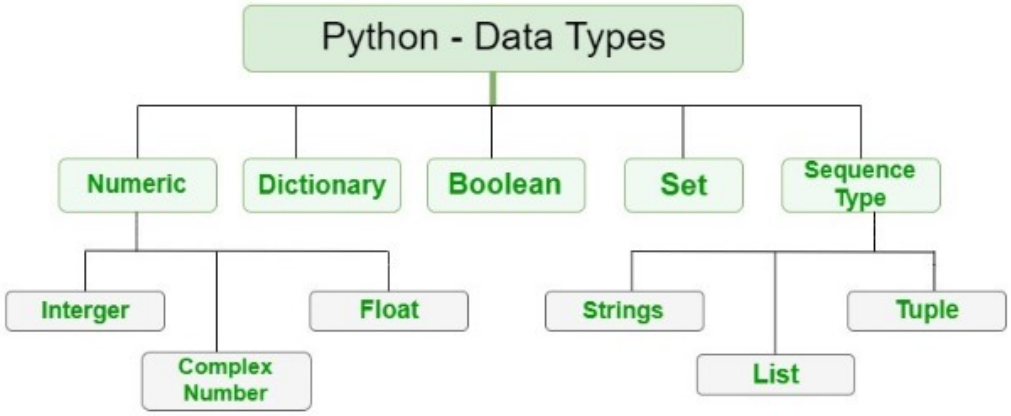
```
print(eval('10+5'))
print(eval('10-5'))
print(eval('10*5'))
print(eval('10/5'))
print(eval('10//5'))
print(eval('10%5'))

O/P:-
15
5
50
2.0
2
0
```

```
value = eval(input("Enter expression: "))
print(value)

O/P:
Enter expression: 5+10
15
```

```
value = eval(input("Enter expression: "))
print(value)

O/P:
Enter expression: 12-2
10
```

Data Types:-

Data Type represent the type of data present inside a variable.
In Python we are not required to specify the type explicitly. Based on value provided, the type will be assigned automatically. Hence Python is Dynamically Typed Language.



Fundamental Data Types in Python:
In Python, the following data types are considered as Fundamental Data types,

1. **Int**
2. **Float**
3. **Complex**
4. **Bool**
5. **Str**

Note: Python contains several inbuilt functions

1. type():- type() is an in-built or pre-defined function in python that is used to check the data type of the variables. The following example depicts the usage.
Example:-

```
emp_id = 11
name = 'Neeraj'
salary = 50000.40
print("emp_id type is: ", type(emp_id))
print("name type is: ", type(name))
print("salary type is: ", type(salary))
```

```
O/P---
emp_id type is:  <class 'int'>
name type is:  <class 'str'>
salary type is:  <class 'float'>
```

2. id():- to get address of object

```
emp_id = 11
name = 'Neeraj'
salary = 50000.40
print("emp_id id is: ", id(emp_id))
print("name id is: ", id(name))
print("salary id is: ", id(salary))

O/P---
emp_id id is:  3146509648432
name id is:  3146515054320
salary id is:  3146510689840
```

3. print():-to print the value

```
emp_id = 11
name = 'Neeraj'
salary = 50000.40
print("My employee id is: ", emp_id)
print("My name is: ", name)
print("My salary is: ", salary)

O/P---
My employee id is:  11
My name is:  Neeraj
My salary is:  50000.4
```

int data type:
The int data type represents values or numbers without decimal values. In python, there is no limit for the int data type. It can store very large values conveniently.

a=10
type(a) O/P:- **<class 'int'>**

Note: In Python 2nd version long data type was existing but in python 3rd version long data type was removed.

We can represent int values in the following ways
1. Decimal form **(bydefault)**
2. Binary form
3. Octal form
4. Hexa decimal form

1. Decimal form(base-10):
It is the default number system in Python. The allowed digits are: 0 to 9
Ex: a =10

2. Binary form(Base-2):
The allowed digits are : 0 & 1
Literal value should be prefixed with 0b or 0B
Eg: a = 0B1111
a =0B123
a=b111

3. Octal Form(Base-8):
The allowed digits are : 0 to 7
Literal value should be prefixed with 0o or 0O.
Ex: a=0o123
a=0o786

4. Hexa Decimal Form(Base-16):
The allowed digits are : 0 to 9, a-f (both lower and upper cases are allowed)
Literal value should be prefixed with 0x or 0X.
Ex: a=0X9FcE
a=0x9aDF

Note: Being a programmer we can specify literal values in decimal, binary, octal and hexa

decimal forms. But PVM will always provide values only in decimal form.

```
# binary data type(Base-2)
x= 0b1111
y= 0B1010

# by default converted into decimal
print(x) # O/P-15
print(y) # O/P-10



# octal data type(Base-8)
x=0o765
y=0O542
# by default converted into decimal
print(x) # O/P-501
print(y) # O/P-354

# decimal data type(Base-10)---default
x=10
y=50
# by default converted into decimal
print(x) # O/P-10
print(y) # O/P-50


# Hexadecimal data type(Base-16)
x=0X8EA
z=0x5eA
# by default converted into decimal
print(x) # O/P-2282
print(y) # O/P-50
```

Base Conversions:--- Python provide the following in-built functions for base conversions

```
# Base Conversions
# bin()
```

```
print(bin(15)) # o/p- 0b1111
print(bin(0o11)) # o/p- 0b1001
print(bin(0X10)) # o/p- 0b10000


# oct()
print(oct(10)) # o/p-0o12
print(oct(0B1111)) # o/p-0o17
print(oct(0X123)) # o/p-0o443

# hex()
print(hex(100)) # o/p-0x64
print(hex(0B111111)) # o/p-0x3f
print(hex(0o12345)) # o/p- 0x14e5
```

Float Data Type in Python:

The float data type represents a number with decimal values. floating-point numbers can also be written in scientific notation. e and E represent exponentiation. where e and E represent the power of 10. For example, the number 2 * 10pow2 is written as 2E2, such numbers are also treated as floating-point numbers.

```
salary = 50.5
print(salary)
print(type(salary))

O/P---
50.5
<class 'float'>
```

Example: Print float values

```
a = 2e2 # 2*10^2  e stands for 10 to the power
b = 2E2 # 2*10^2
c = 2e3 # 2*10^3
d = 2e1
```

```
print(a)
print(b)
print(c)
print(d)
print(type(a))

O/P----
200.0
200.0
2000.0
20.0
<class 'float'>
```

Complex Data Type in python:

The complex data type represents the numbers that are written in the form of a+bj or a-bj, here a is representing a real part of the number and b is representing an imaginary part of the number. The suffix small j or upper J after b indicates the square root of -1. The part “a” and “b” may contain integers or floats.

```
a = 3+5j
b = 2-5.5j
c = 3+10.5j
print(a)
print(b)
print(c)
print()
print("A+B=",a+b)
print("B+C=",b+c)
print("C+A=",c+a)
print("A*B=",a*b)
print("B*C=",b*c)
print("C*A=",c*a)
print("A+B+C=", a+b+c)
print("A/B=",a/b)

O/P---
(3+5j)
(2-5.5j)
(3+10.5j)
```

```
A+B= (5-0.5j)
B+C= (5+5j)
C+A= (6+15.5j)
A*B= (33.5-6.5j)
B*C= (63.75+4.5j)
C*A= (-43.5+46.5j)
A+B+C= (8+10j)
A/B= (-0.6277372262773723+0.7737226277372262j)
```

Boolean data type in Python:

The bool data type represents Boolean values in python. bool data type having only two values are, True and False. Python internally represents, True as 1(one) and False as 0(zero). An empty string (“ ”) represented as False.

Example: Printing bool values:

```
a = True
b = False
print(a)
print(b)
print(a+a)
print(a+b)

O/P---
True
False
2
1
```

None data type in Python:

None data type represents an object that does not contain any value. If any object has no value, then we can assign that object with None type.

Example: Printing None data type:

```
a = None
print(a)
print(type(a))

O/P------
None
<class 'NoneType'>
```

Sequences in python:

Sequences in Python are objects that can store a group of values. The below data types are called sequences.

1. Str---Immutable
2. Bytes (as a list but in range of o to 256 (256 not included))--Immutable
3. Bytearray---- mutable
4. List-----mutable
5. Tuple----Immutable
6. Range

str data type in python:

A string is a data structure in Python that represents a sequence of characters. It is an immutable data type, meaning that once you have created a string, you cannot change it. A group of characters enclosed within single quotes or double quotes or triple quotes is called a string.

```
# string data type
name1 = 'Neeraj'
name2 = "Neeraj"
name3 = """Neeraj"""

O/P---
Neeraj
Neeraj
Neeraj
```

Bytes Data Type in Python:

Bytes data type represents a group of numbers just like an array. It can store values that are from 0 to 256. The bytes data type cannot store negative numbers. To create a byte data type. We need to create a list. The created list should be passed as a parameter to the bytes() function.

Note: The bytes data type is immutable means we cannot modify or change the bytes object. We can iterate bytes values by using for loop.

Example: creating a bytes data type:

```
# creating a bytes data type
x = [15, 25, 150, 4, 15,19]
y = bytes(x)
print(type(y))

O/P---<class 'bytes'>
```

Example: Accessing bytes data type elements using index

```
# Accessing data by using index
x = [15, 25, 150, 4, 15]
y = bytes(x)
print(y[0])
print(y[1])
print(y[2])
print(y[3])
print(y[4])

O/P---
25
150
4
15
```

Example: Printing the byte data type values using for loop

```
# Bytes data type
x = [15, 25, 150, 4, 15]
y = bytes(x)
for i in y:
    print(i)


O/P---
15
25
```

```
150
4
15
```

Example: To check Values must be in range 0,256

```
x = [10, 20, 300, 40, 15]

y = bytes(x)
```

Output: ValueError: bytes must be in range(0, 256)

Example: To check Byte data type is immutable

```
x = [10, 20, 30, 40, 15]

y = bytes(x)

y[0] = 30
```

Output: TypeError: 'bytes' object does not support item assignment

The bytearray data type is the same as the bytes data type, but bytearray is mutable means we can modify the content of bytearray data type. To create a bytearray

1. We need to create a list
2. Then pass the list to the function bytearrray().
3. We can iterate bytearray values by using for loop.

Example: Creating bytearray data type

```
x = [10, 20, 30, 40, 15]
y = bytearray(x)
print(type(y))
```

Example: Accessing bytearray data type elements using index

```
x = [10, 20, 30, 40, 15]

y = bytearray(x)
```

```
print(y[0])
print(y[1])
print(y[2])
print(y[3])
print(y[4])
```

Example: Printing the byte data type values using for loop

```
x = [10, 20, 00, 40, 15]
y = bytearray(x)
for a in y:
print(a)
```

Example: Values must be in the range 0, 256

```
x = [10, 20, 300, 40, 15]
y = bytearray(x)
```

Output: ValueError: bytes must be in range(0, 256)

Example: Bytearray data type is mutable

```
x = [10, 20, 30, 40, 15]
y = bytearray(x)
print("Before modifying y[0] value: ", y[0])
y[0] = 30
print("After modifying y[0] value: ", y[0])
```

List Data Type in Python:

We can create a list data structure by using square brackets []. A list can store different data types. **The list is mutable.**

Tuple Data Type in Python:

We can create a tuple data structure by using parenthesis (). A tuple can store different data types. **A tuple is immutable.**

Set Data Type:

We can create a set data structure by using parentheses symbols (). The set can store the same type and different types of elements.

Dictionary Data Type in Python:

We can create dictionary types by using curly braces {}. The dict represents a group of elements in the form of key-value pairs like a map.

----: Indexing in Python :---

Indexing is the process of accessing an element in a sequence using its position in the sequence (its index).In Python, indexing starts from 0, which means the first element in a sequence is at position 0, the second element is at position 1, and so on. To access an element in a sequence, you can use square brackets [] with the index of the element you want to access.

In Python, indexing refers to the process of accessing a specific element in a sequence, such as a string or list, using its position or index number. Indexing in Python starts at 0, which means that the first element in a sequence has an index of 0, the second element has an index of 1, and so on.

For example, if we have a string "HELLO", we can access the first letter "H" using its index 0 by using the square bracket notation: string[0]

Positive Index	**0**	**1**	**2**	**3**	**4**
	H	**E**	**L**	**L**	**O**
Negative Index	**-5**	**-4**	**-3**	**-2**	**-1**

Python's built-in index() function is a useful tool for finding the index of a specific element in a sequence. This function takes an argument representing the value to search for and returns the index of the first occurrence of that value in the sequence.

If the value is not found in the sequence, the function raises a ValueError. For example, if we have a list [1, 2, 3, 4, 5], we can find the index of the value 3 by calling list.index(3), which will return the value 2 (since 3 is the third element in the list, and indexing starts at 0).

Python Index Examples

The method index() returns the lowest index in the list where the element searched for appears. If any element which is not present is searched, it returns a **ValueError**.

Example:--

```
list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
element = 3
print(list.index(element))

O/P:-
2
```

Example:--(Throws a ValueError)

```
list = [4, 5, 6, 7, 8, 9, 10]
element = 3 # Not in the list
print(list.index(element))

O/P:-
Traceback (most recent call last):
  File "e:\DataSciencePythonBatch\index.py", line 7, in <module>
    print(list.index(element))
ValueError: 3 is not in list
```

Example:--(Index of a string element)

```
list = [1, 'two', 3, 4, 5, 6, 7, 8, 9, 10]
element = 'two'
print(list.index(element))

O/P:-
1
```

What does it mean to return the lowest index?

```
list = [3, 1, 2, 3, 3, 4, 5, 6, 3, 7, 8, 9, 10]
element = 3
print(list.index(element))
O/P:-
0
```

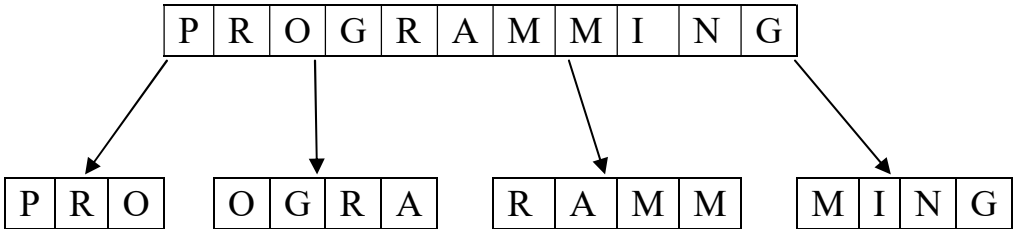
Find element with particular start and end point:--

Syntax:-list_name.index(element, start, stop)

Example:- index() provides you an option to give it hints to where the value searched for might lie.

```
list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
element = 7
print(list.index(element, 5, 8))

O/P:-
6
```

-----: Slicing in Python:-----



Slicing is the extraction of a part of a string, list, or tuple. It enables users to access the specific range of elements by mentioning their indices.

Syntax: Object [start : stop : step]
Object [start : stop]

- **start:** The start parameter in the slice function is used to set the starting position or index of the slicing. The default value of the start is 0.
- **stop:** The stop parameter in the slice function is used to set the end position or index of the slicing[(n-1) for positive value and (n+1) for negative value].
- **step:** The step parameter in the slice function is used to set the number of steps to jump. The default value of the step is 1.

Rules for working :---

Step1:-- Need to check step direction by default it's goes to positive direction.

Setp2:- Need to check start-point and end-point direction.

Step3:-If both directions are matched, then working fine.

Step4:- Otherwise it gives empty subsequence.

-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
I		L	O	V	E		P	Y	T	H	O	N
0	1	2	3	4	5	6	7	8	9	10	11	12

Ex:-1

```
var = "I love python"
print(var[::])

O/P:-
I love python
```

Ex:2

```
var = "I love python"
print(var[::-1])

O/P:-
nohtyp evol I
```

Ex:-3

```
var = "I love python"
print(var[-2:-5:])

O/P:-
```

Ex:-4

```
var = "I love python"
print(var[2:5:-1])

O/P:-
```

Ex:-5

```
var = "I love python"
print(var[::2])

O/P:-
Ilv yhn
```

Ex:-6

```
var = "I love python"
print(var[::-2])
O/P:-
nhy vlI
```

-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
W	E	L	C	O	M	E		T	O		M	Y		B	L	O	G
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Ex:-7,8,9,10,11,12

```
var = "WELCOME TO MY BLOG"
print(var[3:18]) O/P:- COME TO MY BLOG

print(var[2:14:2]) O/P:- LOET Y

print(var[:7]) O/P:- WELCOME

print(var[8:-1:1]) O/P:- TO MY BLO

print(var[-6:-9:-3]) O/P:- Y

print(var[-9:-9:-1]) O/P:-
```

----:String in Python:----

A group of characters enclosed within single or double or triple quotes is called a string. We can say the string is a sequential collection of characters.

```
s1 = "Welcome to 'python' learning"
s2 = 'Welcome to "python" learning'
s3 = """Welcome "to" 'python' learning"""
print(s1)
print(s2)
print(s3)

O/P:--
Welcome to 'python' learning
Welcome to "python" learning
Welcome "to" 'python' learning
```

In-built functions:--

1. len() - To check how many objects/characters present in string.
2. max() -To check which object/character may have maximum ASCII value.
3. min() -To check which object/character may have minimum ASCII value.
4. type() -To check data-type
5. str() -For type casting
6. ord() -Character to ASCII value
7. chr() -ASCII value to character

```
str1="Neeraj"
print(max(str1))
print(min(str1))
print(len(str1))
print(type(str1))

# find ACII value of any charactor(ord() take only one argument)
for i in str1:
    print(ord(i))
```

```
# find ACII value of any special symbol(ord() take only one argument)
x='#'
print("ASCII value of X=",ord(x))
```

```
O/P:--
r
N
6
<class 'str'>
78
101
101
114
97
106
ASCII value of X= 35
```

Accessing string characters in python:

We can access string characters in python by using,

1. Indexing
2. Slicing

Indexing:

Indexing means a position of string's characters where it stores. We need to use square brackets [] to access the string index. String indexing result is string type. String indices should be integer otherwise we will get an error. We can access the index within the index range otherwise we will get an error.

Python supports two types of indexing

1. **Positive indexing:** The position of string characters can be a positive index from left to right direction (we can say forward direction). In this way, the starting position is 0 (zero).
2. **Negative indexing:** The position of string characters can be negative indexes from right to left direction (we can say backward direction). In this way, the starting position is -1 (minus one).

Slicing:--

A substring of a string is called a slice. A slice can represent a part of a string from a string or a piece of string. The string slicing result is string type. We need to use

square brackets [] in slicing. In slicing, we will not get any Index out-of-range exception. In slicing indices should be integer or None or __index__ method otherwise we will get errors.

Different Cases:

wish = “Hello World”

1. wish [::] => accessing from 0th to last
2. wish [:] => accessing from 0th to last
3. wish [0:9:1] => accessing string from 0th to 8th means (9-1) element.
4. wish [0:9:2] => accessing string from 0th to 8th means (9-1) element.
5. wish [2:4:1] => accessing from 2nd to 3rd characters.
6. wish [:: 2] => accessing entire in steps of 2
7. wish [2 ::] => accessing from str[2] to ending
8. wish [:4:] => accessing from 0th to 3 in steps of 1
9. wish [-4: -1] => access -4 to -1

Note: If you are not specifying the beginning index, then it will consider the beginning of the string. If you are not specifying the end index, then it will consider the end of the string. The default value for step is 1

```
wish = "Hello World"
print(wish[::])
print(wish[:])
print(wish[0:9:1])
print(wish[0:9:2])
print(wish[2:4:1])
print(wish[::2])
print(wish[2::])
print(wish[:4:])
print(wish[-4:-1])

O/P:--
Hello World
Hello World
Hello Wor
HloWr
ll
HloWrd
llo World
Hell
orl
```

Strings are immutable in Python:

Once we create an object then the state of the existing object cannot be changed or modified. This behavior is called immutability. Once we create an object then the state of the existing object can be changed or modified. This behavior is called mutability. A string having immutable nature. Once we create a string object then we cannot change or modify the existing object.

```
name = "Python"
print(name)
print(name[0])
name[0]="X"
O/P:-

Python
P
Traceback (most recent call last):
  File "e:\DataSciencePythonBatch\string.py", line 15, in <module>
    name[0]="X"
TypeError: 'str' object does not support item assignment
```

Mathematical operators on string objects in Python

We can perform two mathematical operators on a string. Those operators are:

1. Addition (+) operator.
2. Multiplication (*) operator.

Addition operator on strings in Python:

The + operator works like concatenation or joins the strings. While using the + operator on the string then compulsory both arguments should be string type, otherwise, we will get an error.

```
a = "Python"
b = "Programming"
print(a+b)

O/P:--
```

```
PythonProgramming
```

```
a = "Python"
b = "Programming"
print(a+" "+b)

O/P:--

Python Programming
```

Multiplication operator on Strings in Python:

This is used for string repetition. While using the * operator on the string then the compulsory one argument should be a string and other arguments should be int type.

```
a = "Python"
b = 3
print(a*b)

O/P:--
PythonPythonPython
```

Length of a string in Python:--

We can find the length of the string by using the len() function. By using the len() function we can find groups of characters present in a string. The len() function returns int as result.

```
course = "Python"
print("Length of string is:",len(course))

O/P:--
Length of string is: 6
```

Membership Operators in Python:

We can check, if a string or character is a member/substring of string or not by using the below operators:

1. In
2. not in

in operator:---

in operator returns True, if the string or character found in the main string.

```
print('p' in 'python')
print('z' in 'python')
print('on' in 'python')
print('pa' in 'python')

O/P:--
True
False
True
False
```

```
main=input("Enter main string:")
s=input("Enter substring:")
if s in main:
   print(s, "is found in main string")
else:
   print(s, "is not found in main string")

O/P:--
Enter main string:Neeraj
Enter substring:raj
raj is found in main string
```

Pre-define methods:--

1. **upper()** – This method converts all characters into upper case

```
str1 = 'python programming language'
print('converted to using title():', str1.upper())

str2 = 'JAVA proGramming laNGuage'
```

```
print('converted to using upper():', str2.upper())

str3 = 'WE ARE SOFTWARE DEVELOPER'
print('converted to using upper ():', str3.upper())

O/P:--
converted to using upper(): PYTHON PROGRAMMING LANGUAGE
converted to using upper (): JAVA PROGRAMMING LANGUAGE
converted to using upper (): WE ARE SOFTWARE DEVELOPER
```

2. **lower()** – This method converts all characters into lower case

```
str1 = 'python programming language'
print('converted to using lover():', str1.lower())

str2 = 'JAVA proGramming laNGuage'
print('converted to using lover ():', str2.lower())

str3 = 'WE ARE SOFTWARE DEVELOPER'
print('converted to using lover ():', str3.lower())

O/P:--
converted to using lover(): python programming language
converted to using lover (): java programming language
converted to using lover (): we are software developer
```

3. **swapcase()** – This method converts all lower-case characters to uppercase and all upper-case characters to lowercase

```
str1 = 'python programming language'
print('converted to using swapcase():', str1.swapcase())

str2 = 'JAVA proGramming laNGuage'
print('converted to using swapcase ():', str2.swapcase())

str3 = 'WE ARE SOFTWARE DEVELOPER'
print('converted to using swapcase ():', str3.swapcase())

O/P:--
converted to using title(): PYTHON PROGRAMMING LANGUAGE
converted to using title(): java PROgRAMMING LAngUAGE
```

```
converted to using title(): we are software developer
```

4. **title()** – This method converts all character to title case (The first character in every word will be in upper case and all remaining characters will be in lower case)

```
str1 = 'python programming language'
print('converted to using title():', str1.title())

str2 = 'JAVA proGramming laNGuage'.title()
print('converted to using title():', str2.title())

str3 = 'WE ARE SOFTWARE DEVELOPER'.title()
print('converted to using title():', str3.title())

O/P:--
converted to using title(): Python Programming Language
converted to using title(): Java Programming Language
converted to using title(): We Are Software Developer
```

5. **capitalize()** – Only the first character will be converted to upper case and all remaining characters can be converted to lowercase.

```
str1 = 'python programming language'
print('converted to using capitalize():', str1.capitalize())

str2 = 'JAVA proGramming laNGuage'
print('converted to using capitalize ():', str2.capitalize())

str3 = 'WE ARE SOFTWARE DEVELOPER'
print('converted to using capitalize ():', str3.capitalize())

O/P:--
converted to using capitalize(): Python programming language
converted to using capitalize (): Java programming language
converted to using capitalize (): We are software developer
```

6. **center():-**Python String center() Method tries to keep the new string length equal to the given length value and fills the extra characters using the default character (space in this case).

```
str = "python programming language"

new_str = str.center(40)
# here fillchar not provided so takes space by default.
print("After padding String is: ", new_str)

O/P:--
       python programming language      .
```

```
str = "python programming language"

new_str = str.center(40,'#')
# here fillchar not provided so takes space by default.
print("After padding String is: ", new_str)

O/P:--
######python programming language#######
```

```
str = "python programming language"
new_str = str.center(15,'#')
# here fillchar not provided so takes space by default.
print("After padding String is: ", new_str)

O/P:--
python programming language
```

7. **count():-- count()** function is an inbuilt function in Python programming language that returns the number of occurrences of a substring in the given string.

Syntax: string. Count(substring, start= …., end= ….)
Parameters:

The count() function has one compulsory and two optional parameters.

Mandatory parameter:

substring – string whose count is to be found.

Optional Parameters:

start (Optional) – starting index within the string where the search starts.

end (Optional) – ending index within the string where the search ends.

```
str = "python programming language"
count = str.count('o')
# here fillchar not provided so takes space by default.
print("count of given charactor is: ", count)

O/P:--
ount of given charactor is:  2
```

```
str = "python programming language"
count = str.count('o',5,9)
# here fillchar not provided so takes space by default.
print("count of given charactor is: ", count)

O/P:--
count of given charactor is:  0
```

8. **Join():---** The string join() method returns a string by joining all the elements of an iterable (list, string, tuple), separated by the given separator.

The join() method takes an iterable (objects capable of returning its members one at a time) as its parameter.Some of the example of iterables are: Native data types - List, Tuple, String, Dictionary and Set.

```
str = ['Python', 'is', 'a', 'programming', 'language']
# join elements of text with space
print(' '.join(str))

O/P:--
Python is a programming language
```

```
str = ['Python', 'is', 'a', 'programming', 'language']
# join elements of text with space
print('_'.join(str))

O/P:-
Python_is_a_programming_language
```

```
# .join() with lists
numList = ['1', '2', '3', '4']
separator = ', '
print(separator.join(numList))

O/P:--
1, 2, 3, 4
# .join() with tuples
numTuple = ('1', '2', '3', '4')
print(separator.join(numTuple))
O/P:--
1, 2, 3, 4
s1 = 'abc'
s2 = '123'
# each element of s2 is separated by s1
# '1'+ 'abc'+ '2'+ 'abc'+ '3'
print('s1.join(s2):', s1.join(s2))

O/P:--
s1.join(s2): 1abc2abc3

# each element of s1 is separated by s2
# 'a'+ '123'+ 'b'+ '123'+ 'b'
print('s2.join(s1):', s2.join(s1))

O/P:--
s2.join(s1): a123b123c
```

```
# .join() with sets
test = {'2', '1', '3'}
s = ', '
```

```
print(s.join(test))

O/P:--
2, 3, 1

test = {'Python', 'Java', 'Ruby'}
s = '->-'
print(s.join(test))

O/P:--
Ruby->->Java->->Python
```

```
# .join() with dictionaries
test = {'mat': 1, 'that': 2}
s = '->'

# joins the keys only
print(s.join(test))

O/P:--
mat->that
```

9. **split():--** The split() method splits a string at the specified separator and returns a list of substrings.

```
str = "Python is a programming language"
print(str.split(" "))
str = "Python is a programming language"
print(str.split(",",2))
print(str.split(":",4))
print(str.split(" ",1))
print(str.split(" ",0))

O/P:--
['Python', 'is', 'a', 'programming', 'language']
['Python is a programming language']
['Python is a programming language']
['Python', 'is a programming language']
['Python is a programming language']
```

---: List :---

Whenever we want to create a group of objects where we want below mention properties, then we are using list sequence.

1. Duplicates are allowed.
2. Order is preserved.
3. Objects are mutable.
4. Indexing are allowed.
5. Slicing are allowed.
6. Represented in square bracket with comma separated objects.
7. Homogeneous and Heterogeneous both objects are allowed.

1. Duplicates are allowed.

```
List=['neeraj', 10,20,30,10,20]
print(List)

O/P:--
['neeraj', 10, 20, 30, 10, 20]
```

2. Order is preserved:

```
List=['neeraj', 10,20,30,10,20]
x=0
for i in List:
    print('List[{}] = '.format(x),i)
    x=x+1

O/P:--
List[0] =  neeraj
List[1] =  10
List[2] =  20
List[3] =  30
List[4] =  10
List[5] =  20
```

3. Objects are mutable.

```
List=['neeraj', 10,20,30,10,20]
x=0
for i in List:
    print('List[{}] = '.format(x),i)
    x=x+1
List[0]="Arvind"
print(List)

O/P:--
List[0] =  neeraj
List[1] =  10
List[2] =  20
List[3] =  30
List[4] =  10
List[5] =  20
['Arvind', 10, 20, 30, 10, 20]
```

4. Indexing are allowed.

```
List=['neeraj', 10,20,30,10,20]
print(List[0])
print(List[1])
print(List[2])
print(List[3])
print(List[4])
print(List[5])

O/P:--
neeraj
10
20
30
10
20
```

5. Slicing are allowed:

```
List=['neeraj', 10,20,30,10,20]
print(List[:5])
```

```
O/P:--
['neeraj', 10, 20, 30, 10]


List=['neeraj', 10,20,30,10,20]
print(List[::-1])

O/P:--
[20, 10, 30, 20, 10, 'neeraj']
```

Inbuilt functions in list:

6. **len(list)**
7. **max(list)**
8. **min(list)**
9. **sum(list)**
10. **list(tuple)**
11. **type(list)**

Methos:--

1. **list.append(obj/list/str)-** add object in last

```
animals = ['cat', 'dog', 'rabbit']
# Add 'rat' to the list
animals.append('rat')
print('Updated animals list: ', animals)

O/P:--
Updated animals list:  ['cat', 'dog', 'rabbit', 'rat']

animals = ['cat', 'dog', 'rabbit']
wild_animals = ['tiger', 'fox']
animals.append(wild_animals)
print('Updated animals list: ', animals)

O/P:--
Updated animals list:  ['cat', 'dog', 'rabbit', ['tiger', 'fox']]
```

2. **list.count(obj)** – count how many times given-object are present in list

```
numbers = [2, 3, 5, 2, 11, 2, 7]
count = numbers.count(2)
print('Count of 2:', count)

O/P:--
Count of 2: 3

# vowels list
vowels = ['a', 'e', 'i', 'o', 'i', 'u']
count = vowels.count('i')
print('The count of i is:', count)
count = vowels.count('p')
print('The count of p is:', count)

O/P:--
The count of i is: 2
The count of p is: 0
# random list
random = ['a', ('a', 'b'), ('a', 'b'), [3, 4]]
count = random.count(('a', 'b'))
print("The count of ('a', 'b') is:", count)
count = random.count([3, 4])
print("The count of [3, 4] is:", count)

O/P:--
The count of ('a', 'b') is: 2
The count of [3, 4] is: 1
```

3. **list.extend(list1)** – add list1 in last of list.

```
# create a list
list1 = [2, 3, 5]
list2 = [1, 4]
list1.extend(list2)
print('List after extend():', list1)

O/P:--
List after extend(): [2, 3, 5, 1, 4]
```

```
list = ['Hindi']
tuple = ('Spanish', 'English')
set = {'Chinese', 'Japanese'}
list.extend(tuple)
print('New Language List:', list)
list.extend(set)
print('Newer Languages List:', list)

O/P:--

New Language List: ['Hindi', 'Spanish', 'English']
Newer Languages List: ['Hindi', 'Spanish', 'English', 'Japanese', 'Chinese']
```

4. **list.insert(index,obj)** – insert given object in given index.
5. **list.pop()** – delete last object from given list.
6. **list.remove(obj)** – Remove given object from given list.
7. **list.reverse() –**

```
Example:---
numbers = ['Neeraj',2, 3, 5, 7]
numbers.reverse()
print('Reversed List:', numbers)

O/P:--
Reversed List: [7, 5, 3, 2, 'Neeraj']

Example:----
numbers = ['Neeraj',2, 3, 5, 7]
print(numbers[::-1])

O/P:--
[7, 5, 3, 2, 'Neeraj']

Example:----
numbers = ['Neeraj',2, 3, 5, 7]
# print(numbers[::-1])
list=[]
for i in reversed(numbers):
    list.append(i)
print(list)
```

```
O/P:--
[7, 5, 3, 2, 'Neeraj']
```

8. **list.sort**(reverse=True/False) default-False

```
Example:---
numbers = [2, 3, 7, 5, 4]
numbers.sort()
print('Sort_List:', numbers)

O/P:--
Sort_List: [2, 3, 4, 5, 7]
Example:---
numbers = [2, 3, 7, 5, 4]
numbers.sort(reverse=True)
print('Sort_List:', numbers)

O/P:--
Sort_List: [7, 5, 4, 3, 2]
```

---:Tuple :---

In Python, tuples are immutables. Meaning, you cannot change items of a tuple once it is assigned. There are only two tuple methods count() and index() that a tuple object can call.

1. **Duplicates are allowed.**
2. **Order is preserved.**
3. **Objects are immutable.**
4. **Indexing is allowed.**
5. **Slicing is allowed.**
6. **Represented in parenthesis () with comma separated objects.**
7. **Homogeneous and Heterogeneous both objects are allowed.**

Tuple occupies less memory as compare to list, that's why tuple is more faster as compare to list**.**

Example:--

```
list = [10,20,30,40,50,60,70]
tuple = (10,20,30,40,50,60,70)
print(sys.getsizeof('Size of list = ',list))
print(sys.getsizeof('Size of tuple',tuple))

O/P-  64
      62
```

Built-in functions:-

1. **Len(tuple) # tuple variable must be a iterable.**
2. **Max(tuple)**
3. **Min(tuple)**
4. **Sum(tuple)**
5. **Tuple(list)**
6. **Type(tuple)**

Methods:--

1. **Count(obj). (How many occurrences)**

```
# Creating tuples
Tuple = (0, 1, (2, 3), (2, 3), 1, [3, 2],'Neeraj', (0), (0,))
res = Tuple.count((2, 3))
print('Count of (2, 3) in Tuple is:', res)

res = Tuple.count(0)
print('Count of 0 in Tuple is:', res)

res = Tuple.count((0,))
print('Count of (0,) in Tuple is:', res)

O/P:--
Count of (2, 3) in Tuple is: 2
Count of 0 in Tuple is: 2
Count of (0,) in Tuple is: 1
```

2. **Index(obj,start,stop)(obj is compulsory argument but rest are optional)**

```
Tuple = (0, 1, 2, 3, 2, 3, 1, 3, 2)
# getting the index of 3
res = Tuple.index(3)
print(res)
O/P:--
3
Tuple = (0, 1, 2, 3, 2, 3, 1, 3, 2)
# getting the index of 3
print(Tuple.index(3,4))
O/P:--
5
Tuple = (0, 1, 2, 3, 2, 3, 1, 3, 2)
# getting the index of 3
print(Tuple.index(3,0,4))
o/p:--
3
```

---: Set :---

If we want to represent a group of unique elements then we can go for sets. Set cannot store duplicate elements.

1. **Duplicates are not allowed.**
2. **Order is not preserved.**
3. **Objects are mutable.**
4. **Indexing is not allowed.**
5. **Slicing is not allowed.**
6. **Represented in { } with comma separated objects.**
7. **Homogeneous and Heterogeneous both objects are allowed.**

```
# Creating a set
s = {10,20,30,40}
print(s)
print(type(s))

O/P:--
{40, 10, 20, 30}
<class 'set'>

# Creating a set with different elements
s = {10,'20','Rahul', 234.56, True}
print(s)
print(type(s))

O/P:--
{'20', True, 234.56, 10, 'Rahul'}
<class 'set'>

# Creating a set using range function
s=set(range(5))
print(s)

O/P:--
{0, 1, 2, 3, 4}
```

```
# Duplicates not allowed
s = {10, 20, 30, 40, 10, 10}
print(s)
print(type(s))
O/P:--
{40, 10, 20, 30}
<class 'set'>

# Creating an empty set
s=set()
print(s)
print(type(s))

O/P:--
set()
<class 'set'>
```

 # Methods in set:----

1. **add(only_one_argument not iterable)**

```
s={10,20,30,50}
s.add(40)
print(s)

O/P:--
{40, 10, 50, 20, 30}
```

2. **update(iterable_obj1,iterable_obj2)**

```
s = {10,20,30}
l = [40,50,60,10]
s.update(l)
print(s)

O/P:--
{40, 10, 50, 20, 60, 30}


s = {10,20,30}
```

```
l = [40,50,60,10]
s.update(l, range(5))
print(s)

O/P:--
{0, 1, 2, 3, 4, 40, 10, 50, 20, 60, 30}
```

Difference between add() and update() methods in set:

1. We can use add() to add individual items to the set, whereas we can use update() method to add multiple items to the set.
2. The add() method can take one argument whereas the update() method can take any number of arguments but the only point is all of them should be iterable objects.

3. **copy() --Clone of set**

```
s={10,20,30}
s1=s.copy()
print(s1)

O/P:--
{10, 20, 30}
```

4. **pop()---** This method removes and returns some random element from the set.

```
s = {40,10,30,20}
print(s)
print(s.pop())
print(s)

O/P:--
{40, 10, 20, 30}
40
{10, 20, 30}
```

5. **remove(element) ---** This method removes specific elements from the set. If the specified element is not present in the set then we will get KeyError.

```
s={40,10,30,20}
s.remove(30)
```

```
print(s)

O/P:--
{40, 10, 20}

s={40,10,30,20}
s.remove(50)
print(s)

O/P:--
Traceback (most recent call last):
  File "E:\DataSciencePythonBatch\sets.py", line 65, in <module>
    s.remove(50)
KeyError: 50
```

6. **discard(element) ---** This method removes the specified element from the set. If the specified element is not present in the set, then we won't get any error.

```
s={10,20,30}
s.discard(10)
print(s)

O/P:--
{20, 30}

s={10,20,30}
s.discard(40)
print(s)

O/P:--
{10, 20, 30}
```

7. **clear() --- removes all elements from the set.**

```
s={10,20,30}
print(s)
s.clear()
print(s)
```

```
O/P:--
{10, 20, 30}
set()
```

MATHEMATICAL OPERATIONS ON SETS

1. **union() ---** This method return all elements present in both sets.

```
x={10,20,30,40}
y={30,40,50,60}
print(x.union(y))

O/P:--
{40, 10, 50, 20, 60, 30}
```

2. **intersection() ---** This method returns common elements present in both x and y.

```
x = {10,20,30,40}
y = {30,40,50,60}
print(x.intersection(y))
print(x&y)
print(y.intersection(x))
print(y&x)

O/P:--
{40, 30}
{40, 30}
{40, 30}
{40, 30}
```

3. **difference() ---** This method returns the elements present in x but not in y

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
z = x.difference(y)
print(z)
```

O/P:--- {‘banana’, ‘cherry’}

---: Dictionary :---

If we want to represent a group of objects as key-value pairs then we should go for dictionaries.

Characteristics of Dictionary

1. Dictionary will contain data in the form of key, value pairs.
2. Key and values are separated by a colon ":" symbol
3. One key-value pair can be represented as an item.
4. Duplicate keys are not allowed.
5. Duplicate values can be allowed.
6. Heterogeneous objects are allowed for both keys and values.
7. Insertion order is not preserved.
8. Dictionary object having mutable nature.
9. Dictionary objects are dynamic.
10. Indexing and slicing concepts are not applicable

syntax for creating dictionaries with key,value pairs is: **d = { key1:value1, key2:value2, …., keyN:valueN }**

Creating an Empty dictionary in Python:

```
d = {}
print(d)
print(type(d))

O/P:--
{}
<class 'dict'>
```

Adding the items in empty dictionary:--

```
d = {}
d[1] = "Neeraj"
d[2] = "Rahul"
d[3] = "Ravi"
print(d)

O/P:--
{1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
```

Accessing dictionary values by using keys:--

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}

print(d[1])
print(d[2])
print(d[3])

O/P:--
Neeraj
Rahul
Ravi
```

Note:--- While accessing, if the specified key is not available then we will get **KeyError**

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}

print(d[1])
print(d[2])
print(d[3])
print(d[10])

O/P:--
Neeraj
Rahul
Ravi
Traceback (most recent call last):
  File "E:\DataSciencePythonBatch\dict.py", line 16, in <module>
    print(d[10])
KeyError: 10
```

handle this KeyError by using in operator:

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}

if 10 in d:
    print(d[10])
else:
    print('Key Not found')
```

```
O/P:--
Key Not found
```

Getting student information's in the form of dictionaries:--

```
d={}
n=int(input("Enter how many student detail you want: "))
i=1
while i <=n:
   name=input("Enter Employee Name: ")
   email=input("Enter Employee salary: ")
   d[name]=email
   i=i+1
print(d)

O/P:--
Enter how many student detail you want: 3
Enter Employee Name: Neeraj
Enter Employee salary: neeraj@gmail.com
Enter Employee Name: Rahul
Enter Employee salary: rahul@gmail.com
Enter Employee Name: Ravi
Enter Employee salary: ravi@gmail.com
{'Neeraj': 'neeraj@gmail.com', 'Rahul': 'rahul@gmail.com', 'Ravi':
'ravi@gmail.com'}
```

Updating dictionary elements:

We can update the value for a particular key in a dictionary. The syntax is:

d[key] = value

Case1: While updating the key in the dictionary, if the key is not available then a new key will be added at the end of the dictionary with the specified value.

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
print("Old dict data",d)
d[10]="Arvind"
print("Nwe dict data",d)
```

```
O/P:--
Old dict data {1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
Nwe dict data {1: 'Neeraj', 2: 'Rahul', 3: 'Ravi', 10: 'Arvind'}
```

Case2: If the key already exists in the dictionary, then the old value will be replaced with a new value.

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
print("Old dict data",d)
d[2]="Arvind"
print("New dict data",d)

O/P:--
Old dict data {1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
New dict data {1: 'Neeraj', 2: 'Arvind', 3: 'Ravi'}
```

Removing or deleting elements from the dictionary:

1. By using the del keyword, we can remove the keys
2. By using clear() we can clear the objects in the dictionary

By using the del keyword

Syntax: del d[key]

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
del d[3]
print("New dict is",d)

O/P:--
New dict is {1: 'Neeraj', 2: 'Rahul'}
```

By using clear() keyword

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
d.clear()
print("New dict is",d)

O/P:--
New dict is {}
```

Delete entire dictionary object:- We can also use the del keyword to delete the total dictionary object. Before deleting we just have to note that once it is deleted then we cannot access the dictionary.

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
del d
print("New dict is",d) O/P:--

Traceback (most recent call last):
  File "E:\DataSciencePythonBatch\dict.py", line 51, in <module>
    print("New dict is",d)
NameError: name 'd' is not defined. Did you mean: 'id'?
```

Functions and methods of dictionary in Python

1. dict() function
2. len() function
3. clear() method
4. get() method
5. pop() method
6. popitem() method
7. keys() method
8. Items() method
9. copy() method

dict() function:

This can be used to create an empty dictionary.

```
d=dict()
print(d)
print(type(d))

O/P:--
{}
<class 'dict'>
```

len() function: This function returns the number of items in the dictionary.

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
```

```
print(len(d))

O/P:--
3
```

clear() method: This method can remove all elements from the dictionary.

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
print(d.clear())O/P:--

O/P:--
None
```

get() method:

This method used to get the value associated with the key. This is another way to get the values of the dictionary based on the key. The biggest advantage it gives over the normal way of accessing a dictionary is, this doesn't give any error if the key is not present. Let's see through some examples:

Case1: If the key is available, then it returns the corresponding value otherwise returns None. It won't raise any errors.
Syntax: d.get(key)

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
print(d.get(1))
print(d.get(2))
print(d.get(3))

O/P:--
Neeraj
Rahul
Ravi
```

Case 2: If the key is available, then returns the corresponding value otherwise returns the default value that we give.
Syntax: d.get(key, defaultvalue)

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
```

```
print(d.get(7,"Neeraj"))
print(d.get(6,"Neeraj"))
print(d.get(5,"Neeraj"))

O/P:--
Neeraj
Neeraj
Neeraj
```

pop() method: This method removes the entry associated with the specified key and returns the corresponding value. If the specified key is not available, then we will get KeyError.

Syntax: d.pop(key)

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
d.pop(3)
print(d)

O/P:
{1: 'Neeraj', 2: 'Rahul'}
```

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi'}
print(d.pop(3)) O/P:-- Ravi
```

popitem() method: This method removes an arbitrary item(key-value) from the dictionary and returns it.

```
d={1: 'Neeraj', 2: 'Rahul', 3: 'Ravi',4:'Jai',5:'Santosh'}
print(d.popitem())
print(d)

O/P:--
(5, 'Santosh')
{1: 'Neeraj', 2: 'Rahul', 3: 'Ravi', 4: 'Jai'}
```

keys() method: This method returns all keys associated with the dictionary

```
d = {1: 'Ramesh', 2: 'Suresh', 3: 'Mahesh'}
print(d)
for k in d.keys():
   print(k)

O/P:--
1
2
3
```

values() method: This method returns all values associated with the dictionary

```
d = {1: 'Ramesh', 2: 'Suresh', 3: 'Mahesh'}
print(d)
for k in d.values():
   print(k)


O/P:--
Ramesh
Suresh
Mahesh
```

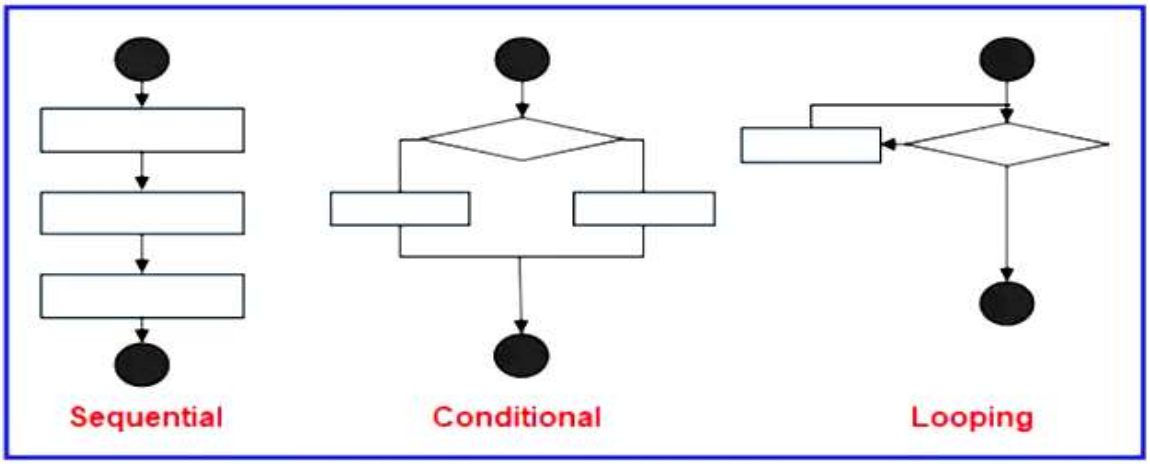
items() method: A key-value pair in a dictionary is called an item. items() method returns the list of tuples representing key-value pairs.

```
d = {1: 'Ramesh', 2: 'Suresh', 3: 'Mahesh'}
for k, v in d.items():
   print(k, "---", v)

O/P:--
1 --- Ramesh
2 --- Suresh
3 --- Mahesh
```

Control Flow Statements

In programming languages, flow control means the order in which the statements or instructions, that we write, get executed. In order to understand a program, we should be aware of what statements are being executed and in which order. So, understanding the flow of the program is very important. There are, generally, three ways in which the statements will be executed. They are,

1. **Sequential**
2. **Conditional**
3. **Looping**



Sequential: In this type of execution flow, the statements are executed one after the other sequentially. By using sequential statements, we can develop simple programs

Example: Sequential statements:-

```
print("Welcome")
print("to")
print("python class")

O/P:-
Welcome
to
python class
```

Conditional: Statements are executed based on the condition. As shown above in the flow graph, if the condition is true then one set of statements are executed, and if false then the other set. Conditional statements are used much in complex programs.

Conditional statements are also called decision-making statements. Let's discuss some conditions making statements in detail. There are three types of conditional statements in python. They are as follows:

1. **if statement**
2. **if-else statement**
3. **nested-if (if-elif-elif-else)**

if-statement:-
syntax:-

```
if condition:
    print("Block statement")
print("Out of block statement")
```

Example:-

```
num=int(input("Enter any no: "))
if num>=18:
    print("if block statment executed")
print("out of if block statements")

O/P:
Enter any no: 10
out of if block statements
PS E:\DataSciencePythonBatch> python control.py
Enter any no: 18
if block statment executed
out of if block statements
```

Example:---

```
# Example:---Checking if a number is positive, negative, or zero.

num = float(input("Enter a number: "))
if num > 0:
    print("The number is positive.")
elif num < 0:

    print("The number is negative.")
else:
    print("The number is zero.")
```

if-else condition:-
syntax:-

```

if condition:
    print("if block statement executed")
else:
    print("else block statement executed ")
```

Example:---

```
num=int(input("Enter any no: "))
if num>=18:
    print("if block statment executed")
else:
    print("else block statement executed")

O/P:-
PS E:\DataSciencePythonBatch> python control.py
Enter any no: 18
if block statment executed
PS E:\DataSciencePythonBatch> python control.py
Enter any no: 15
else block statement executed
```

Example:---

```
# Example:---Find grater no.

x=int(input("Enter first no."))
y=int(input("Enter second no."))
z=int(input("Enter third no."))
if x>y:
    if x>z:
        print("Greater ni is(x): ",x)
    else:
        print("Greater no is(z): ",z)
else:
    if y>z:
        print("Greater ni is(y): ",y)
    else:
        print("Greater no is(z): ",z)

O/P:-

Enter first no.10
Enter second no.10
Enter third no.20
Greater no is(z):  20
```

Example:---

```
# Example:-- Checking if a person is eligible to vote

age = int(input("Enter your age: "))
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")

O/P:-
Enter your age: 35
You are eligible to vote.
```

Example:---

```
# Example:-- Checking if a year is a leap year

year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0 and year %100==0):
    print("It's a leap year.")
else:
    print("It's not a leap year.")

O/P:-
Enter a year: 2000
It's a leap year.
```

Nested-If else:--

```
# Example:-- Check your gread based on your own score

score = int(input("Enter your score: "))

if score >= 90:
    print("You got an A.")
else:
    if score >= 80:
        print("You got a B.")
    else:
        if score >= 70:
            print("You got a C.")
        else:
            if score >= 60:
                print("You got a D.")
            else:
                print("You got an F.")

O/P:-
Enter your score: 90
You got an A.
```

Example:--

```
# Example:-- Check given year is leep year or not.

year = int(input("Enter a year: "))
if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
            print("Leap year")
        else:
            print("Not a leap year")
    else:
        print("Leap year")
else:
    print("Not a leap year")
```

if elif else statement in python:

Syntex:

```
if (condition1):
    statement of if Block
elif(condition2):
    statment of elif Block
elif(condition3):
    statement if elif block
else:
    ststement of else block
```

Example:---

```
# example:-- Please choose value within range of o to 4.

print("Please enter the values from 0 to 4")
x=int(input("Enter a number: "))
if x==0:
      print("You entered:", x)
elif x==1:
      print("You entered:", x)
```

```
elif x==2:
      print("You entered:", x)
elif x==3:
      print("You entered:", x)
elif x==4:
      print("You entered:", x)
else:
      print("Beyond the range than specified")

O/P:-
Enter a number: 5
Beyond the range than specified
PS E:\DataSciencePythonBatch> python control.py
Please enter the values from 0 to 4
Enter a number: 4
You entered: 4
```

Example:- Python Program to calculate the square root.

```
# Example:-Python

num = float(input('Enter a number: '))
num_sqrt = num ** 0.5
print('The square root of Num :', num_sqrt)

O/P:-
Enter a number: 4
The square root of 4.000 is 2.000
PS E:\DataSciencePythonBatch> python control.py
Enter a number: 8
The square root of 8.000 is 2.828
```

Example:-- Python Program to find the area of triangle.

```
# Python Program to find the area of triangle
# s = (a+b+c)/2
# area = √(s(s-a)*(s-b)*(s-c))
a = float(input('Enter first side: '))
b = float(input('Enter second side: '))
c = float(input('Enter third side: '))
s = (a + b + c) / 2
area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
print('The area of the triangle is :', area)

O/P:---
Enter first side: 5
Enter second side: 6
Enter third side: 7
The area of the triangle is : 14.696938456699069
```

Example:-- Python program to swap two variables.

```
# Python program to swap two variables
x = input('Enter value of x: ')
y = input('Enter value of y: ')

# create a temporary variable and swap the values
temp = x
x = y
y = temp
print('The value of x after swapping: {}'.format(x))
print('The value of y after swapping: {}'.format(y))

O/P:---
Enter value of x: 5
Enter value of y: 8
The value of x after swapping: 8
The value of y after swapping: 5


# without using third variable
x = input('Enter value of x: ')
```

```
y = input('Enter value of y: ')
x, y = y, x
print('The value of x after swapping: {}'.format(x))
print('The value of y after swapping: {}'.format(y))

O/P:---
Enter value of x: 4
Enter value of y: 6
The value of x after swapping: 6
The value of y after swapping: 4
# By-using Addition and Subtraction.
x = int(input('Enter value of x: '))
y = int(input('Enter value of y: '))
x = x + y
y = x - y
x = x - y
print('The value of x after swapping: {}'.format(x))
print('The value of y after swapping: {}'.format(y))

O/P:---
Enter value of x: 4
Enter value of y: 6
The value of x after swapping: 6
The value of y after swapping: 4
# By-using Multiplication and division.
x = int(input('Enter value of x: '))
y = int(input('Enter value of y: '))
x = x * y
y = x / y
x = x / y
print('The value of x after swapping: {}'.format(x))
print('The value of y after swapping: {}'.format(y))

O/P:---
Enter value of x: 2
Enter value of y: 5
The value of x after swapping: 5.0
The value of y after swapping: 2.0
```

```
# By-using x-or(^) operator.
x = int(input('Enter value of x: '))
y = int(input('Enter value of y: '))
x = x ^ y
y = x ^ y
x = x ^ y
print('The value of x after swapping: {}'.format(x))
print('The value of y after swapping: {}'.format(y))

O/P:--
Enter value of x: 10
Enter value of y: 20
The value of x after swapping: 20
The value of y after swapping: 10
```

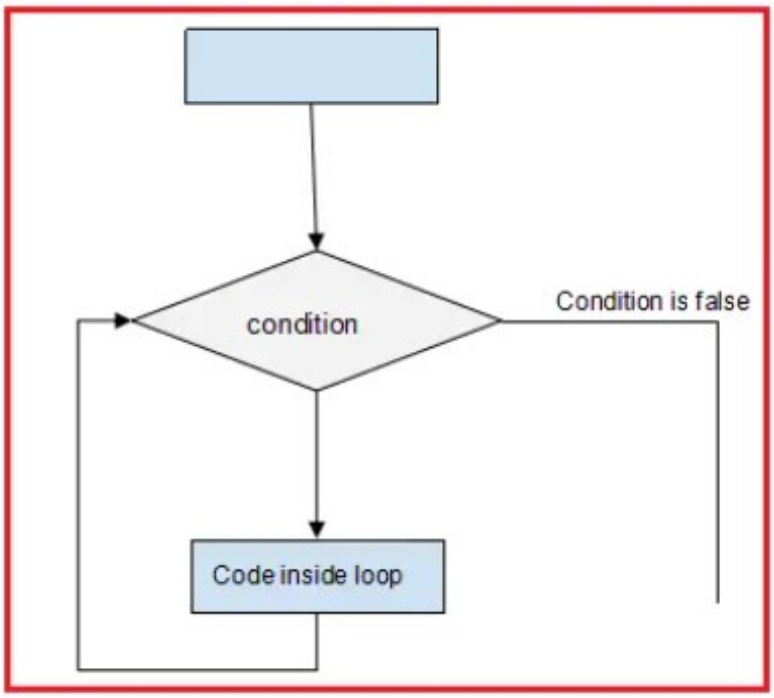
----:LOOPING Statement in Python (Iterations):----

If we want to execute a group of statements multiple times, then we should go for a looping kind of execution. There are two types of loops available in python.
They are:

1. **while loop**
2. **for loop**

1. while loop:- The while loop contains an expression/condition. As per the syntax colon (:) is mandatory otherwise it throws a syntax error. The condition gives the result as bool type, either True or False. The loop keeps on executing the statements until the condition becomes False. i.e. With the while loop we can execute a set of statements as long as a condition is true.

Flowchart for the while-loop:



Parts of while loop in Python:
Initialization:

This is the first part of the while loop. Before entering the condition section, some initialization is required.

Condition:
Once the initializations are done, then it will go for condition checking which is the heart of the while loop. The condition is checked and if it returns True, then execution enters the loop for executing the statements inside.

After executing the statements, the execution goes to the increment/decrement section to increment the iterator. Mostly, the condition will be based on this iterator value, which keeps on changing for each iteration. This completes the first iteration. In the second iteration, if the condition is False, then the execution comes out of the loop else it will proceed as explained in the above point.

Increment/Decrement section: This is the section where the iterator is increased or decreased. Generally, we use arithmetic operators in the loop for this section.

Example: Printing numbers from 1 to 5 by using while loop

1. The program is to print the number from 1 to 5
2. Before starting the loop, we have made some assignments(x = 1). This is called the Initialization section.
3. After initialization, we started the while loop with a condition x<=5. This condition returns True until x is less than 5.
4. Inside the loop, we are printing the value of x.
5. After printing the x value, we are incrementing it using the operator x+=1. This is called the increment/decrement section.
6. For each iteration, the value of x will increase and when the x value reaches 6, then the condition x<=5 returns False. At this iteration, the execution comes out of the loop without executing the statements inside. Hence in the output '6' is not printed.

```
x=1
while x<=5:
    print(x)
    x+=1
```

```
O/P:--
1
2
3
4
5
```

```
# Printing numbers from 1 to 5 by using while loop.
x=1
while x<=5:
    print(x)
    x+=1

# Printing numbers from 1 to 5 by using while loop.
x=1
while x<=5:
    if x<5:
        print(x,end=",")
    else:
        print(x,end="")
    x+=1


# Printing even numbers from 10 to 20 by using while loop.
x=10
while (x>=10) and (x<=20):
   print(x)
   x+=2
print("End")

# print sun of given n netural no
x=int(input("Enter any no : "))
sum=0
i=1
while i<=x:
    sum=sum+i
    if i<x:
        print(i,end="+")
```

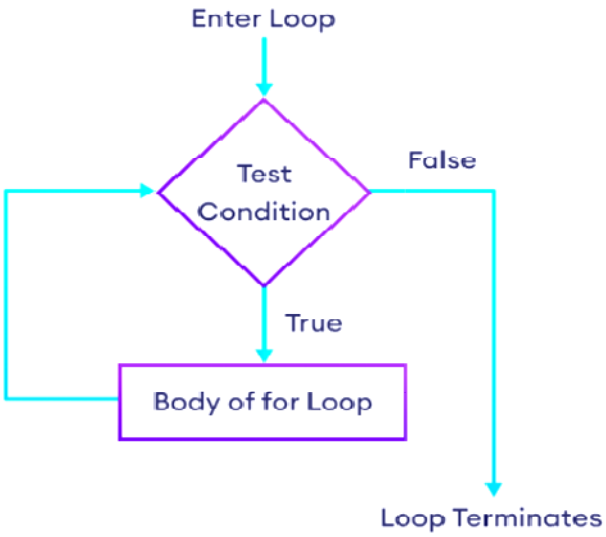
```
    else:
        print(i,end="=")
    i=i+1
print(sum)


# print n even numbers
x= int(input("Enter how many even number you want :"))
n=1
while n<=x:
    print(2*n)
    n=n+1

# print n even numbers(1,2,3,4,5,6,--------)
x= int(input("Enter how many even numbers you want :"))
n=1
while n<=x:
    if n<x:
        print(2*n,end=",")
    else:
        print(2*n,end="")
    n=n+1


# Print sum of given even numbers.
x= int(input("Enter how many even numbers sum you want :"))
n=1
sum=0
while n<=x:
    sum=sum+2*n
    if n<x:
        print(2*n,end="+")
    else:
        print(2*n,end="=")
    n=n+1
print(sum)


# Print n odd numbers
x= int(input("Enter how many odd number you want :"))
n=1
```

```
while n<=x:
    if n<x:
        print((2*n-1),end=",")
    else:
        print((2*n-1),end="")
    n=n+1

# Print sum of n odd numbers
x= int(input("Enter how many odd number you want :"))
n=1
sum = 0
while n<=x:
    sum=sum+(2*n-1)
    if n<x:
        print((2*n-1),end="+")
    else:
        print((2*n-1),end="=")
    n=n+1
print(sum)
```

for loop in python:

Basically, a for loop is used to iterate elements one by one from sequences like string, list, tuple, etc. This loop can be easily understood when compared to the while loop. While iterating elements from the sequence we can perform operations on every element.

The Python For Loop is used to repeat a block of statements until there are no items in the Object may be String, List, Tuple, or any other object.



1. Initialization: We initialize the variable(s) here. Example i=1.
2. Items in Sequence / Object: It will check the items in Objects. For example, individual letters in String word. If there are items in sequence (True), then it will execute the statements within it or inside. If no item is in sequence (False), it will exit.
3. After completing the current iteration, the controller will traverse to the next item.
4. Again it will check the new items in sequence. The statements inside it will be executed as long as the items are in sequence.

Range() function :----

The Python range function helps to generate a sequence of numbers. Or this Python range function helps to iterate items in iterables such as Lists, Tuples, Sets, Strings, etc.

The syntax of this Python function contains range start, stop, step. All the arguments such as start, stop, and step accepts positive or Negative integers.
Syntax: range(start, stop, step)

- Start(optional) – Starting position number. If you omit this, the Python range function will start from 0.
- Stop – A value before this number is the end value. For example, (1, 10) print values from 1 to 9.
- Step (optional) – Sequence of numbers generated. It determines the space or difference between each integer value. For example, (1, 10, 2) returns 1, 3, 5, 7, and 9. You can notice that the difference between each integer is 2.

Python's built-in range() function is mainly used when working with for loops – you can use it to loop through certain blocks of code a specified number of times. The range() function accepts three arguments – one is required, and two are optional. By default, the syntax for the range() function looks similar to the following:

range(stop)---The stop argument is **required**.

The range() function returns a sequence of numbers starting from 0, incrementing by 1, and ending at the value you specify as stop (non-inclusive). But what if you want to iterate through a range of two numbers you specify and don't want to start the counting from 0? You can pass a second **optional** start argument, start, to specify the starting number. The syntax to do so looks like this:

range(start, stop)

This syntax generates a sequence of numbers based on the start (inclusive) and stop (non-inclusive) values that increment by 1.

Lastly, if you don't want the default increment to be 1, you can specify a third **optional** argument, step. The syntax to do that looks like this:

range(start, stop, step)

```
x=range(11)
print(x)
print(list(x))
print(tuple(x))
print(set(x))
```

```
O/P:--
range(0, 11)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

Transfer Statements:

1. Break
2. continue
3. pass

Break statement:--- We can use break statement inside loops to break loop execution based on some condition.

```
for i in range(10):
    if i==7:
        print("processing is enough..  plz break !!!!!!! ")
        break
    print(i)

O/P:---
0
1
2
3
4
5
6
processing is enough..  plz break !!!!!!!

list=[10,20,600,60,70]
for i in list:
    if i>500:
        print("no need to check next object of list")
        break
    print(i)

O/P:--
10
20
no need to check next object of list
```

continue statement:-- We can use continue statement to skip current iteration and continue next iteration.

```
for i in range(10):
    if i%2==0:
        continue
    print(i)

O/P:--
1
3
5
7
9

list=[10,20,600,60,70]
for i in list:
    if i>500:
        print("no need to print this object")
        continue
    print(i)

O/P:--
10
20
no need to print this object
60
70

list=[10,20,600,60,70]
for i in list:
    if i>500:
        continue
    print(i)

O/P:--
10
20
60
70
```

pass statement:--
pass is a keyword in Python. In our programming syntactically if block is required which won't do anything then we can define that empty block with pass keyword.

1. It is an empty statement
2. It is null statement
3. It won't do anything

```
for i in range(100):
    if i%9==0:
        print(i)
    else:
        pass

O/P:--
0
9
18
27
36
45
54
63
72
81
90
99
```

----: Some Important Pattern examples :---

```
# O/P:--
# 10
# 10 8
# 10 8 6
# 10 8 6 4
# 10 8 6 4 2
# code for above output
k=[]
for i in range(10,1,-2):
    k.append(i)
    for j in k:
        print(j,end=" ")
    print()


# O/P:-- {1: 2, 2: 3}
# code for above output
t=(1,1,2,2,4,2)
dict={}
for i in t:
    count=0
    for j in t:
        if j==i:
            count=count+1
    if count>=2:
        dict[i]=count
print(dict)

#  O/P --- {1: 2, 2: 3, 4: 1}
# code for above output
t=(1,1,2,2,4,2)
dict={}
for i in t:
    count=0
    for j in t:
        if j==i:
            count=count+1
    dict[i]=count
print(dict)
```

```
# *
# **
# ***
# ****
# *****
n=int(input("Enter the number of rows: "))
for i in range(1,n+1):
    print("*"*i)


# *
# * *
# * * *
# * * * *
# * * * * *
n=int(input("Enter the number of rows: "))
for i in range(1,n+1):
    print("* "*i)


#      *
#     **
#    ***
#   ****
#  *****

n=int(input("Enter the number of rows: "))
for i in range(1,n+1):
    print(" "*(n-i),"*"*i)


#      *
#     ***
#    *****
#   *******
#  *********

n=int(input("Enter the number of rows: "))
for i in range(1,n+1):
    print(" "*(n-i),"*"*(2*i-1))
```

```
#       *
#      * *
#     * * *
#    * * * *
#   * * * * *
n=int(input("Enter the number of rows: "))
for i in range(1,n+1):
    print(" "*(n-i)," *"*i)


# 1
# 1 2
# 1 2 3
# 1 2 3 4
# 1 2 3 4 5

n=int(input("Enter the number of rows:"))
for i in range(1,n+1):
    for j in range(1,i+1):
        print(j,end=" ")
    print()


#  *****
#   ****
#    ***
#     **
#      *
n=int(input("Enter the number of rows: "))
for i in range(n,0,-1):
    print(" "*(n-i),"*"*i)

# *****
# ****
# ***
# **
# *
n=int(input("Enter the number of rows: "))
for i in range(n,0,-1):
    print("*"*i)
```

```
#  * * * * *
#   * * * *
#    * * *
#     * *
#      *
n=int(input("Enter the number of rows: "))
for i in range(n,0,-1):
    print(" "*(n-i)," *"*i)



#      *
#     * *
#    * * *
#   * * * *
#  * * * * *
#  * * * * *
#   * * * *
#    * * *
#     * *
#      *
n=int(input("Enter the number of rows: "))
for i in range(1,n+1):
    print(" "*(n-i),"* "*i)
m=n-1
for i in range(m,0,-1):
    print(" "*(m-i)," *"*i)




#      *
#     **
#    ***
#   ****
#  *****
# *****
# ****
```

```
# ***
# **
# *

n=int(input("Enter the number of rows: "))
for i in range(0,n+1):
    print(" "*(n-i),"*"*i)
for i in range(n,0,-1):
    print("*"*i," "*(n-i))


# *
# * *
# * * *
# * * * *
# * * * * *
# * * * *
# * * *
# * *
# *
n=int(input("Enter the number of rows: "))
for i in range(0,n+1):
    print("* "*i)
m=n-1
for i in range(m,0,-1):
    print("* "*i)




#      *
#     **
#    ***
#   ****
#  *****
#  *****
#   ****
#    ***
#     **
#      *
```

```
n=int(input("Enter the number of rows: "))
for i in range(0,n+1):
    print(" "*(n-i),"*"*i)
for i in range(n,0,-1):
    print(" "*(n-i),"*"*i)
```

IF-ELIF-ELSE STATEMENT EXAMPLES

Example 1: Write a program to check given no is positive. (Only if-statement)
Example 2: Write a program to check given no is positive or negative. (Only if-else statement)
Example 3: Write a program to check given no is positive, negative or Zero.(Only if-elif-else statement)
Example 4: Write a program to swap two variables without using third variable.
Example 5: Write a program to swap two variables using third variable.
Example 6: Write a program to swap two variables using using Addition and Subtraction.
Example 7: Write a program to swap two variables using Multiplication and division.
Example 8: Write a program to swap two variables using x-or(^) operator.
Example 9: Write a program to find squre root of given no.
Example 10: Write a program to find largest no among the three inputs numbers.
Example 11: Write a program to find area of triangle. (1/2* hight*base)
Example 12: Write a program to find area of square, rectangle.
Example 13: Write a program to find given year is leep year or not.

While-Loop EXAMPLES

Example 1: Write a program to display n natural numbers. (In Horizontal-1,2,3,4,5……..)
Example 2: Write a program to calculate the sum of numbers.
Example 3: Write a program to find even no. (2,4,6,8,….)
Example 4: Write a program find odd no.(1,3,5,7,9,……)
Example 5: Write a program to find factorial of given no.
Example 6: Write a program to print your names ten times.
Example 7: Write a program to find how many vowels and consonants are present in strings.
Example 8: Write a program to add 5 in each elements in given list. [10,20,30,40,50]
Example 9: Write a program to add 5 in each elements in given tuple. (10,20,30,40,50)
Example 10: Write a program to create a list from given string.

Examples for FOR_LOOPs

Example 1: Print the first 10 natural numbers using for loop.
Example 2: Python program to print all the even numbers within the given range.
Example 3: Python program to calculate the sum of all numbers from 1 to a given number.
Example 4: Python program to calculate the sum of all the odd numbers within the given range.
Example 5: Python program to print a multiplication table of a given number
Example 6: Python program to display numbers from a list using a for loop.
Example 7: Python program to count the total number of digits in a number.
Example 8: Python program to check if the given string is a palindrome.**(madam=madam)**
Example 9: Python program that accepts a word from the user and reverses it.
Example 10: Python program to check if a given number is an Armstrong number**. (153=1**3+5**3+3**3)**
Example 11: Python program to count the number of even and odd numbers from a series of numbers.
Example 12: Python program to display all numbers within a range except the prime numbers.
Example 13: Python program to get the Fibonacci series. (0,1,1,2,3,5,8,13,21……..)
Example 14: Python program to find the factorial of a given number.
Example 15: Python program that accepts a string and calculates the number of digits and letters.
Example 16: Write a Python program that iterates the integers from 1 to 25.
Example 17: Python program to check the validity of password input by users.
Example 18: Python program to convert the month name to a number of days.

----:Functions:----

If a group of statements is repeatedly required then it is not recommended to write these statements every time separately. We have to define these statements as a single unit and we can call that unit any number of times based on our requirement without rewriting.This unit is nothing but function.

```
x=10
y=20
print("Addition of x & y =",x+y)
print("Addition of x & y =",x-y)
print("Addition of x & y =",x*y)

x=200
y=100
print("Addition of x & y =",x+y)
print("Addition of x & y =",x-y)
print("Addition of x & y =",x*y)

x=10
y=5
print("Addition of x & y =",x+y)
print("Addition of x & y =",x-y)
print("Addition of x & y =",x*y)

O/P:--
Addition of x & y = 30
Addition of x & y = -10
Addition of x & y = 200
Addition of x & y = 300
Addition of x & y = 100
Addition of x & y = 20000
Addition of x & y = 15
Addition of x & y = 5
Addition of x & y = 50
```

Now, repeated code can be bound into single unit that is called function.

The advantages of function:

1. **Maintaining the code is an easy way.**
2. **Code re-usability.**

Now,

```
def calculate(x, y):
    print("Addition of x & y =",x+y)
     print("Addition of x & y =",x-y)
     print("Addition of x & y =",x*y)

calculate(10,20)
calculate(200,100)
calculate(10,5)

O/P:--
Addition of x & y = 30
Addition of x & y = -10
Addition of x & y = 200
Addition of x & y = 300
Addition of x & y = 100
Addition of x & y = 20000
Addition of x & y = 15
Addition of x & y = 5
Addition of x & y = 50
```

Types of function:---

1. **In-built function :--** The functions which are coming along with Python software automatically,are called built-in functions or pre defined functions.
   Examples:--
   1. print()
   2. id()
   3. type()
   4. len()
   5. eval()
   6. sorted()
   7. count() etc…….

2. User define function:--- The functions which are defined by the developer as per the requirement are called user-defined functions.

Syntax:---

```
def fun_name(parameters….):
  ‘‘‘ doc string….’’’

  Statment1…….
  Statment2…….
  Statment3…….

  return (anything)
# call function
fun_name(arguments....)
```

Important terminology	
def-keyword	mandatory
return-keyword	optional
arguments	optional
parameters	optional
fun_name	mandatory

1. **def keyword –** Every function in python should start with the keyword ‘def’. In other words, python can understand the code as part of a function if it contains the ‘def’ keyword only.
2. **Name of the function –** Every function should be given a name, which can later be used to call it.
3. **Parenthesis** – After the name ‘()’ parentheses are required
4. **Parameters** – The parameters, if any, should be included within the parenthesis.
5. **Colon symbol ‘:’** should be mandatorily placed immediately after closing the parentheses.
6. **Body –** All the code that does some operation should go into the body of the function. The body of the function should have an indentation of one level with respect to the line containing the ‘def’ keyword.
7. **Return statement** – Return statement should be in the body of the function. It’s not mandatory to have a return statement. If we are not writing return statement then default return value is **None**
8. **Arguments**:-- At the time of calling any function, in between the parentheses we passes arguments.

Relation between parameters and arguments:--

When we are creating a function, if we are using parameters in between parenthesis, then it is compulsory to at the time of calling this function, you need to pass correspond arguments.

Parameters are inputs to the function. If a function contains parameters, then at the time of calling, compulsory we should provide values as a arguments, otherwise we will get error.

```
def calculate(x, y):
    print("Addition of x & y =",x+y)
    print("Addition of x & y =",x-y)
    print("Addition of x & y =",x*y)

calculate(10,20)
calculate(200,100)
calculate(10,5)

O/P:--
Addition of x & y = 30
Addition of x & y = -10
Addition of x & y = 200
Addition of x & y = 300
Addition of x & y = 100
Addition of x & y = 20000
Addition of x & y = 15
Addition of x & y = 5
Addition of x & y = 50
```

```
# Write a function to take number as input and print its square value
def square(x):
    print("The Square of",x,"is", x*x)
square(4)
square(5)

O/P:--
The Square of 4 is 16
The Square of 5 is 25
```

```
# Write a function to check whether the given number is even or odd?
def even_odd(num):
    if num%2==0:
        print(num,"is Even Number")
    else:
        print(num,"is Odd Number")

even_odd(10)
even_odd(15)

O/P:--
10 is Even Number
15 is Odd Number
```

```
# Write a function to find factorial of given number?
def fact(num):
    result=1
    while num>=1:
        result=result*num
        num=num-1
    return result
i=int(input("Enter any no "))
print("The Factorial of",i,"is :",fact(i))

O/P:--
Enter any no 5
The Factorial of 5 is : 120
```

Returning multiple values from a function: In other languages like C, C++ and Java, function can return almost one value. But in Python, a function can return any number of values.

```
def add_sub(a,b):
    add=a+b
    sub=a-b
    return add,sub

x,y=add_sub(100,50)
```

```
print("The Addition is :",x)
print("The Subtraction is :",y)

O/P:--
The Addition is : 150
The Subtraction is : 50

          Or

def add_sub(a,b):
    add=a+b
    sub=a-b
    return add,sub
x,y=int(input("Enter first value:")),int(input("Enter second value: "))
print("The Addition is :",x)
print("The Subtraction is :",y)

O/P:--
The Addition is : 100
The Subtraction is : 50
```

```
def calc(a,b):
    add=a+b
    sub=a-b
    mul=a*b
    div=a/b
    return add,sub,mul,div

x,y,z,p=calc(int(input("Enter first value:")),int(input("Enter second value: ")))
print("The Addition is",x)
print("The Substraction is",y)
print("The Multip is",z)
print("The Division is",p)
O/P:--
Enter first value:100
Enter second value: 10
The Addition is 110
The Substraction is 90
The Multip is 1000
The Division is 10.0
```

Types of arguments:

```
def f1(a,b):
        ------
        ------
f1(10,20)
```

There are 5 types are actual arguments are allowed in Python.

1. **positional arguments:**

```
def f1(a,b):
        ------
        ------
f1(10,20)
```

```
def square(x):
   print("The Square of",x,"is", x*x)
square(4)
square(5)

O/P:-
The Square of 4 is 16
The Square of 5 is 25
```

2. **keyword arguments:**

```
def f1(a,b):
        ------
        ------
f1(a=10,b=20)
```

```
def square(x):
   print("The Square of",x,"is", x*x)
square(x=4)
square(x=5)

O/P:--
The Square of 4 is 16
The Square of 5 is 25
```

3. **default arguments:**

```
def f1(a=0,b=0):
        ------
        ------
f1(10,20)
f1()
```

```
def square(x=0):
    print("The Square of",x,"is", x*x)
square(x=4)
square()

O/P:--
The Square of 4 is 16
The Square of 0 is 0
```

4. **Variable length arguments:**

```
def f1(*n):
        ------
        ------
f1(10)
f1(10,20)
f1(10,20,30)
```

```
def sum(*n):
    total=0
    for i in n:
        total=total+i
    print("The Sum=",total)
sum()
sum(10)
sum(10,20)
sum(10,20,30,40)

O/P:--
The Sum= 0
The Sum= 10
The Sum= 30
The Sum= 100
```

5. **key word variable length arguments:**

```
def f1(**n):
    ------
    ------
f1(n1=10, n2=20)
```

```
def display(**kwargs):
    for k,v in kwargs.items():
        print(k,"=",v)
display(n1=10,n2=20,n3=30)
print("-----------")
display(rno=100, name="Neeraj", marks=70, subject="Java")

O/P:--
n1 = 10
n2 = 20
n3 = 30
-----------
rno = 100
name = Neeraj
marks = 70
subject = Java
```

Types of Variables in Python

The variables based on their scope can be classified into two types:

1. **Local variables**
2. **Global variables**

Local Variables in Python:

The variables which are declared inside of the function are called local variables. Their scope is limited to the function i.e we can access local variables within the function only. If we are trying to access local variables outside of the function, then we will get an error.

```
def a():
    x=10
    return "value of Local variable is:",x

def b():
    return "value of Local variable is:",x

p=a()
print(p)
y=b()
print(y)

O/P:-
('value of Local variable is:', 10)
Traceback (most recent call last):
  File "E:\Python Core_Advance\local.py", line 10, in <module>
    y=b()
  File "E:\Python Core_Advance\local.py", line 6, in b
    return "value of Local variable is:",x
NameError: name 'x' is not defined
```

We Can’t access local variable outside the function:

```
def a():
    x=10
    return "value of Local variable is:",x

def b():
    return "value of Local variable is:",x

p=a()
print(p)
print(x)

O/P:--
('value of Local variable is:', 10)
Traceback (most recent call last):
  File "E:\Python Core_Advance\local.py", line 12, in <module>
    print(x)
NameError: name 'x' is not defined
```

Global variables in Python:

The variables which are declared outside of the function are called global variables. Global variables can be accessed in all functions of that module.

```
a=11
b=12
def m():
   print("a from function m(): ",a)
   print("b from function m(): ",b)
def n():
   print("a from function n(): ",a)
   print("b from function n(): ",b)
m()
n()


O/P:--
a from function m():  11
b from function m():  12
a from function n():  11
b from function n():  12
```

GLOBAL KEYWORD IN PYTHON

The keyword global can be used for the following 2 purposes:

1. To declare a global variable inside a function
2. To make global variables available to the function.

```
def m1():
   global a
   a=2
   print("a value from m1() function: ", a)
def m2():
   print("a value from m2() function:", a)
m1()
m2()

O/P:--
a value from m1() function: 2
a value from m2() function: 2
```

global and local variables having the same name in Python

```
a=1
def m1():
  global a
  a=2
  print("a value from m1() function:", a)
def m2():
  print("a value from m2() function:", a)
m1()
m2()


O/P:--
a value from m1() function: 2
a value from m2() function: 2
```

If we use the global keyword inside the function, then the function is able to read-only global variables.
PROBLEM: This would make the local variable no more available.

globals() built-in function in python:
The problem of local variables not available, due to the use of global keywords can be overcome by using the Python built-in function called globals(). The globals() is a built-in function that returns a table of current global variables in the form of a dictionary. Using this function, we can refer to the global variable "a" as: global()["a"].

```
a=1
def m1():
  a=2
  print("a value from m1() function:", a)
  print("a value from m1() function:", globals()['a'])

m1()

O/P:--
a value from m1() function: 2
a value from m1() function: 1
```

---: Higher order function :---

A function in Python with another function as an argument or returns a function as an output is called the High order function. A function that is having another function as an argument or a function that returns another function as a return in the output

- The function can be stored in a variable.
- The function can be passed as a parameter to another function.
- The high order functions can be stored in the form of lists, hash tables, etc.
- Function can be returned from a function.

1. **Map()**
2. **Filter()**
3. **Lambda()**
4. **Reduce()**
5. **Decorators()**
6. **Generators()**

---: Map :----

Python's map() is a built-in function that enables the processing and transformation of all items in an iterable without the need for an explicit for loop, a technique referred to as mapping. This function is particularly useful when you want to apply a transformation function to each element in an iterable, producing a new iterable as a result. map() is one of the tools that facilitate a functional programming approach in Python.

map() Syntax

map(function, iterable, ...)

map() Arguments

The map() function takes two arguments:

1. function - a function
2. iterable - an iterable like sets, lists, tuples, etc

The map() function returns an object of map class. The returned value can be passed to functions like list() - to convert to list, set() - to convert to a set, and so on.

Example:-1

```
# Map() higher order function----------

my_list=[10,20,30,40]

def sqr(n):
    return n*n

x=map(sqr,my_list)
print(x)
print(list(x))

O/P:--
<map object at 0x000001EA310E3490>
[100, 400, 900, 1600]
```

Example:-2

```
my_tuple=(10,20,30,40)

def sqr(n):
    return n*n

x=map(sqr,my_tuple)
print(x)
print(tuple(x))

O/P:-
<map object at 0x0000019833A83490>
(100, 400, 900, 1600)
```

Example:-3

```
my_str="Neeraj"
def add(n):
    x=ord(n)
    return x
x=map(add,my_str)
print(x)
print(list(x))

O/P:-
<map object at 0x000001D03A4E3490>
[78, 101, 101, 114, 97, 106]
```

Example:-4

```
my_str="Neeraj"
def add(n):
    x=ord(n)
    return chr(x+5)
x=map(add,my_str)
print(x)
print(list(x))

O/P:-
<map object at 0x0000026D8F1634C0>
['S', 'j', 'j', 'w', 'f', 'o']
```

---: Filter :---

The filter function extracts elements from an iterable (such as a list or tuple) based on the results of a specified function. This function is applied to each element of the iterable, and if it returns True that element is included in the output of the filter() function.

filter() Syntax

The syntax of filter() is: **filter(function, iterable)**

filter() Arguments

The filter() function takes two arguments:

1. **function** - a function
2. **iterable** - an iterable like sets, lists, tuples etc.

The filter() function returns an iterator.

Example 1:-

```
# filter() higher order function ----------------
my_list=[60,10,70,90,55,75,10,20,40]

def fun(n):
    if n>=60:
        return True

x=filter(fun , my_list)
print(list(x))

O/P:--
[60, 70, 90, 75]
```

Example 2:-

```
def check_even(number):
    if number % 2 == 0:
        return True
    return False

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers_iterator = filter(check_even, numbers)
even_numbers = list(even_numbers_iterator)
print(even_numbers)

O/P:--
[2, 4, 6, 8, 10]
```

Example 3:-

```
def check_odd(number):
    if number % 2 != 0:
        return True
    return False

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
odd_numbers_iterator = filter(check_odd, numbers)
odd_numbers = list(odd_numbers_iterator)
print(odd_numbers)

O/P:--
[1, 3, 5, 7, 9]
```

---: Lambda :---

a lambda function is a special type of function without the function name. For example, lambda : print('Hello World').

A lambda function can take any number of arguments, but can only have one expression.

Here, we have created a lambda function that prints 'Hello World'.

lambda Function Declaration: We use the **lambda keyword** instead of def to create a lambda function. Here's the syntax to declare the lambda function:

Syntex:----

lambda argument(s) : expression

argument(s) - any value passed to the lambda function

expression - expression is executed and returned

Let's see an example,

```
# without argument
greet = lambda : print('Hello World')
greet()

O/P:--
Hello World
```

greet = lambda : print('Hello World'). Here, we have defined a lambda function and assigned it to the variable named greet. In the above example, we have defined a lambda function and assigned it to the greet variable. When we call the lambda function, the print() statement inside the lambda function is executed.

```
# with argument
x=lambda p,q,r:3*p+4*q+5*r+5
print(x(10,20,30))
O/P:-
265
```

```
# with argument
user = lambda name : print('Hello', name)
user('Neeraj')

O/P:--
Hello Neeraj
```

---: Reduce :---

The reduce() function in Python is part of the functools module, which needs to be imported before it can be used.

This function performs functional computation by taking a function and an iterable (such as a list, tuple, or dictionary) as arguments. It applies the function cumulatively to the elements of the iterable, reducing it to a single value. Unlike other functions that may return multiple values or iterators, reduce() returns a single value, which is the result of the entire iterable being condensed into a single integer, string, or boolean.

Steps of how to reduce function works:

1. The function passed as an argument is applied to the first two elements of the iterable.
2. After this, the function is applied to the previously generated result and the next element in the iterable.
3. This process continues until the whole iterable is processed.
4. The single value is returned as a result of applying the reduce function on the iterable.

```
from functools import reduce

def product(x,y):
    return x*y

ans = reduce(product, [2, 5, 3, 7])
print(ans)

O/P:--
210
```

```
import functools

my_list=(10,20,60,30,40)
def greater(a,b):
    if a>b:
        return a
    else:
        return b
x=functools.reduce(greater,my_list)
print(x)

O/P:-
60

my_list=(10,20,60,30,40)
def lowest_digit(a,b):
    if a<b:
        return a
    else:
        return b
x=functools.reduce(lowest_digit,my_list)
print(x)

O/P:-
10

my_str="Neeraj"
def greater(a,b):
    if a>b:
        return a
    else:
        return b
x=functools.reduce(greater,my_str)
print("This char have greater asci value:",x)

O/P:-
This char have greater asci value: r
```

Practice Questions for higher order functions

```
Example:1
x = lambda p,q,r: print(p+q+r)
x(2,4,6)

Example:2
my_list = [1,2,3,4,5,6,7,8,9,10]
def squr1(x):
    return x**2
x = lambda my_list:print(list(map(squr1,my_list)))
x(my_list)

Example:3
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
print(squared)

Example:4
a = [1, 2, 3]
b = [4, 5, 6]
summed = list(map(lambda x, y: x + y, a, b))
print(summed)

Example:5
words = ['hello', 'world']
uppercased = list(map(lambda s: s.upper(), words))
print(uppercased)

Example:6
p = lambda arg: [arg * x for x in range(1, 5)]
print(p(10))


Example:7
p = lambda n: [x for x in range(1, n+1) if x%2==0]
print(p(10))
```

```
Example:8
p = lambda n: [x for x in range(1, n+1) if x%2!=0]
n = int(input("Enter how many odd no you want"))
print(p(n))

Example:9
squared = list(map(lambda x: x ** 2, range(5)))
print(squared)

Example:10
even_numbers = list(filter(lambda x: x % 2 == 0, range(10)))
print(even_numbers)

Example:11
cubed = list(map(lambda x: x ** 3, range(1, 6)))
print(cubed)

Example:12
cubed = [(lambda x: x ** 3)(x) for x in range(1, 6)]
print(cubed)


# ----------------- if-else --------------------
Example:1

lambda arguments: value_if_true if condition else value_if_false
check_even_odd = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check_even_odd(5))
print(check_even_odd(8))

Example:2
max_value = lambda x, y: x if x > y else y
print(max_value(10, 20))

Example:3
numbers = list(range(10))
categorized = list(map(lambda x: "Even" if x % 2 == 0 else "Odd", numbers))
print(categorized)
```

Example:4

```
grade = lambda x: 'A' if x >= 90 else ('B' if x >= 80 else ('C' if x >= 70 else 'F'))
print(grade(85))
print(grade(95))
print(grade(65))
```